

PHP Senior

100 вопросов и ответов для собеседования

Язык и runtime, архитектура, базы данных, распределённые системы, security, testing, observability, reliability и инженерное лидерство.

Ответы построены как тезис -> механизм -> trade-off -> практическая проверка.

Содержание

1. PHP language/runtime internals	1–10
2. OOP, типизация и принципы проектирования	11–20
3. Composer, стандарты и внутреннее устройство framework-подходов	21–30
4. Базы данных, SQL, транзакции и ORM	31–40
5. Архитектура, DDD, модульный монолит и CQRS	41–50
6. Распределённые системы, highload, очереди и caching	51–60
7. Безопасность	61–70
8. Тестирование, статический анализ и качество	71–80
9. Производительность, наблюдаемость, DevOps и надёжность	81–90
10. Лидерство, инциденты и практический system design	91–100

Как пользоваться: сначала сформулируйте ответ за 60–90 секунд, затем сверяйтесь с разбором и отдельно проговаривайте ограничения решения.

1. PHP language/runtime internals

1. Что на самом деле разделяется между запросами в PHP-FPM, а что создаётся заново, и почему это важно при проектировании приложения?

Ответ: PHP-FPM использует долгоживущие worker-процессы, но каждый HTTP-запрос получает новый request context Zend Engine. Пользовательские переменные, объекты, открытые на уровне запроса ресурсы и обычные static-переменные логически живут до request shutdown; затем движок вызывает shutdown-функции и освобождает request-allocated memory.

Между запросами остаются сам процесс и его расширения, подключённые библиотеки, OPcache с байткодом, preloaded-классы, persistent connections и нативные аллокации расширений. Поэтому shared-nothing не означает «новый процесс»: утечка в C-расширении, состояние persistent-соединения или рост глобального кеша расширения переживают запросы.

Архитектурное следствие — не рассчитывать на PHP static как на межзапросный кеш и одновременно не считать worker стерильным. Persistent-соединение нужно проверять перед использованием, секреты нельзя оставлять в process-wide storage, а рост RSS ограничивают наблюдаемостью и pm.max_requests, который является предохранителем, а не лечением утечки.

Слабый сигнал: Утверждение, что каждый запрос PHP-FPM всегда запускает новый процесс; Попытка хранить межзапросное состояние в static-свойстве пользовательского класса; Использование pm.max_requests как единственного способа разобраться с ростом памяти.

2. Как устроены zval, refcount и copy-on-write, и в каких случаях «дешёвое копирование» внезапно становится дорогим?

Ответ: Переменная PHP представлена zval: в ней находятся тип и либо само скалярное значение, либо указатель на refcounted-структуру, например строку, массив или объект. При присваивании массива или строки движок обычно увеличивает refcount и применяет copy-on-write: физическая копия появляется только при первой модификации одного из владельцев.

Объекты ведут себя иначе: копируется zval с тем же object handle, а не состояние объекта; независимый объект создаёт clone. Явные ссылки через & помещают переменные в reference container и меняют семантику присваивания, поэтому они могут неожиданно связывать значения и усложнять оптимизации.

Дорогой случай — передать большой массив по значению, а затем изменить его хотя бы в одном месте: HashTable дублируется целиком. Передача по ссылке «для ускорения» часто ошибочна: до записи COW уже дешёв, а ссылка добавляет связанность. Для больших потоков данных лучше менять форму API: генератор, DTO, батчи или специализированная структура.

```
1  $a = range(1, 1_000_000);
2  $b = $a;                      // пока разделяется один HashTable
3  $b[0] = 42;                   // здесь отделяется и копируется массив
4
5  $service = new stdClass();
6  $alias = $service;            // тот же object handle
7  $copy = clone $service;       // новый объект
```

PHP

Слабый сигнал: Фраза «PHP всегда копирует массив при присваивании» без упоминания COW; Фраза «объекты передаются по ссылке» вместо объяснения object handle; Повсеместные параметры по ссылке ради мнимой экономии памяти.

3. Как PHP собирает циклический мусор и почему unset() не гарантирует немедленного снижения RSS процесса?

Ответ: Refcount освобождает обычный граф, как только число владельцев становится нулём, но цикл может сохранить положительные счётчики после потери внешней ссылки. Для таких кандидатов Zend GC ведёт root buffer и периодически запускает алгоритм поиска циклов; запуск можно наблюдать через gc_status() и инициировать gc_collect_cycles().

unset() удаляет ссылку из таблицы символов, но не обязан уничтожить значение, если владельцы остались. Даже после уничтожения PHP memory manager часто сохраняет освобождённые страницы в арене процесса для повторного использования, поэтому memory_get_usage() может упасть, а RSS в ОС — нет.

В долгоживущем queue worker важно разрывать ссылки на большие графы, очищать Identity Map/контейнеры пакетной обработки и измерять как PHP memory, так и RSS. Ручной `gc_collect_cycles()` оправдан в контролируемой точке после крупного батча, но вызов после каждой итерации повышает CPU и маскирует удерживающую ссылку.

```

1  final class Node {
2      public ?Node $next = null;
3  }
4
5  $a = new Node();
6  $b = new Node();
7  $a->next = $b;
8  $b->next = $a;
9  unset($a, $b);
10
11 $collected = gc_collect_cycles();

```

PHP

Слабый сигнал: Отождествление `unset()` с немедленным возвратом памяти операционной системе; Отключение GC без профилирования и доказанной причины; Принудительный сбор циклов в каждой итерации горячего цикла.

4. Как безопасно провести upgrade production-приложения с PHP 8.4 на PHP 8.5?

Ответ: Сначала фиксируют совместимость вне production: проверяют Composer platform requirements и нативные extensions, включают 8.5 в CI, прогоняют unit/integration/contract и нагрузочные тесты, статический анализ и логи deprecation на 8.4. Затем изучают не только новые возможности, но и incompatible changes: код может проходить синтаксическую проверку и менять warning, coercion или поведение расширения.

Собирают отдельный воспроизводимый runtime image и сравнивают ini, FPM, JIT/preload и расширения. В PHP 8.5 OPcache встроен в binary, поэтому старую инструкцию загрузки zend_extension и packaging assumptions нужно пересмотреть. Rollout делают canary-этапами с метриками ошибок, latency, RSS и возможностью отката всего image вместе с vendor, а не только PHP binary.

Новые конструкции — pipe operator `|>`, `#[NoDiscard]` и closures в constant expressions — внедряют отдельным изменением после успешного runtime upgrade. Это сохраняет диагностируемость и не поднимает минимальную версию пакета случайно; библиотеке заранее обновляют composer.json, CI matrix и SemVer-политику.

```

1  $normalized = $input
2      |> trim(...)
3      |> strtolower(...);
4
5  #[\NoDiscard]
6  function persist(
7      Order $order,
8      OrderRepository $repository
9  ): Result {
10     // Неиспользованный результат вызовет warning;
11     // (void) persist(...) — явное игнорирование.
12     return $repository->save($order);
13 }

```

PHP

Слабый сигнал: Одновременный upgrade runtime и массовый рефакторинг на новые конструкции; Проверка только unit-тестами без extensions, интеграций и canary; Старая конфигурация загрузки OPcache, механически перенесённая на PHP 8.5.

5. Что дают Fibers и почему сами по себе они не делают блокирующий PHP-код асинхронным?

Ответ: Fiber — переключаемый пользователем стек: выполнение можно приостановить `Fiber::suspend()` и продолжить `resume()/throw()`. Это примитив кооперативной многозадачности, а не scheduler и не параллельный поток; в каждый момент PHP-код выполняется на одном системном потоке.

Асинхронность появляется только вместе с event loop и неблокирующим драйвером I/O. Если внутри fiber вызвать обычный блокирующий PDO-запрос, `file_get_contents()` к медленной сети или `sleep()`, заблокируется весь loop, а не только fiber.

Fibers полезны библиотекам, которые скрывают callback/promise plumbing и дают последовательный стиль кода. На границах нужны отмена, deadline, backpressure и гарантированный cleanup в finally; иначе удобный синтаксис создаёт неограниченную конкурентность и утечки ресурсов.

```
1  $fiber = new Fiber(function (): string {
2      $value = Fiber::suspend('need-data');
3      return strtoupper($value);
4  });
5
6  $signal = $fiber->start();           // need-data
7  $fiber->resume('payload');
8  $result = $fiber->getReturn();      // PAYLOAD
```

PHP

Слабый сигнал: Утверждение, что Fiber автоматически выполняется параллельно; Блокирующий I/O внутри event loop без специального адаптера; Запуск неограниченного количества fibers без backpressure и отмены.

6. Почему `memory_limit`, `memory_get_usage()` и RSS показывают разные значения и как диагностировать память долгоживущего worker?

Ответ: `memory_limit` ограничивает память, учитываемую Zend memory manager, но не обязательно все mmap-области и нативные аллокации расширений. `memory_get_usage(false)` показывает используемые блоки PHP, `true` — объём, запрошенный менеджером у ОС; RSS включает также код, shared pages с особенностями учёта, стеки и память библиотек.

Рост `allocated` при стабильном `used` часто означает фрагментацию или сохранение арен для повторного использования. Одновременный рост `used` указывает на удерживаемый граф PHP-объектов, а рост RSS при стабильных PHP-метриках — повод исследовать расширения, драйверы, изображения, FFI и внешние библиотеки.

Диагностика строится по сериям метрик на границах job: `used/allocated`, RSS, число объектов доменных кешей, GC runs/roots, размер батча. Затем уменьшают воспроизводимый сценарий и профилируют; перезапуск worker по лимиту задач остаётся последней защитой от деградации, но не заменяет поиск причины.

Слабый сигнал: Вывод об отсутствии утечки только по `memory_get_usage()`; Сравнение RSS и `memory_limit` как эквивалентных метрик; Бесконечный queue worker без лимитов, телеметрии и стратегии graceful restart.

7. Почему PHP array дорог по памяти и что происходит при переходе packed array в mixed array?

Ответ: PHP array — упорядоченный HashTable, одновременно играющий роль списка и map. Для каждого элемента он хранит `zval` и bucket с hash/key metadata, поэтому миллион целых занимает существенно больше восьми мегабайт; точная величина зависит от версии, архитектуры и формы таблицы.

Последовательные целочисленные ключи позволяют packed representation без отдельного hash index. Строковый ключ, разреженный большой индекс или некоторые мутации переводят структуру в mixed representation; удаление элементов также может оставлять holes. `array_values()` перепакует ключи, но создаст новый массив и пиковое потребление памяти.

Сначала меняют алгоритм: потоковый генератор, обработка батчами, SQL-агрегация или компактное бинарное представление обычно важнее микровыбора контейнера. `SplFixedArray` экономит не всегда достаточно и проигрывает в удобстве; выбор подтверждают измерением на целевой версии PHP.

```
1  $rows = [10, 20, 30];           // packed
2  $rows['total'] = 60;             // mixed HashTable
3
4  $sparse = [];
5  $sparse[1_000_000] = true;      // это map, не массив на миллион слотов
6
7  $packedAgain = array_values($rows); // новая аллокация
```

PHP

Слабый сигнал: Оценка памяти PHP-массива как количество элементов, умноженное на размер скаляра; Безусловная рекомендация SplFixedArray без benchmark; array_values() большого массива без учёта двойного пика памяти.

8. Какие нетривиальные свойства генераторов важны при построении потоковых pipeline?

Ответ: Генератор ленив: тело начинает выполняться при первом rewind/current, а не при вызове функции. yield сохраняет локальное состояние, send() передаёт значение обратно в точку yield, throw() внедряет исключение, yield from делегирует и возвращает return value вложенного генератора через getReturn().

Ленивость не гарантирует потоковость источника. PDOStatement может уже буферизовать весь результат в драйвере, а iterator_to_array() снова материализует поток. rewind() допустим, пока генератор остаётся на первом yield; после продвижения дальше попытка перемотки завершится исключением, поэтому API должно явно обещать одноразовое потребление.

Ресурс открывают внутри try и закрывают в finally: finally выполнится при завершении или уничтожении генератора, но откладывать cleanup до GC рискованно. Потребитель должен полностью исчерпать итератор либо явно прекратить его жизненный цикл; ошибки из producer не следует терять за абстракцией pipeline.

```
1  function lines(string $path): Generator {
2      $handle = fopen($path, 'rb');
3      if ($handle === false) {
4          throw new RuntimeException('Cannot open file');
5      }
6
7      try {
8          while (($line = fgets($handle)) !== false) {
9              yield rtrim($line, "\r\n");
10             }
11         } finally {
12             fclose($handle);
13         }
14     }
```

PHP

Слабый сигнал: Обещание O(1) памяти без проверки буферизации драйвера; Материализация generator через iterator_to_array() в конце pipeline; Ресурс в генераторе без finally.

9. Как различаются Error, Exception, engine warning и фатальные ошибки с точки зрения единой стратегии обработки?

Ответ: Exception и большинство ошибок движка, включая TypeError, ParseError и ValueError, реализуют Throwable; Error не наследует Exception. Поэтому catch (Exception) не является универсальной границей процесса, а catch (Throwable) уместен на верхнем уровне HTTP kernel или consumer loop, где ошибку логируют и преобразуют в ответ либо статус job.

set_error_handler() вызывается для уровней, указанных при его регистрации, даже если текущая маска error_reporting() их подавляет. Если обработчик должен уважать @ и динамическую маску, он проверяет error_reporting() внутри callback; для не обслуживаемых уровней возвращает false. Последнюю фатальную ошибку можно увидеть в shutdown handler через error_get_last(), но продолжить безопасное выполнение после исчерпания памяти обычно нельзя.

Не следует ловить Throwable вокруг произвольной бизнес-операции и продолжать с частично изменённым состоянием. Граница обязана откатить транзакцию, освободить ресурсы, добавить correlation context и решить, ошибка retryable или terminal; пользователю возвращают стабильный контракт без stack trace.

PHP

```
1  set_error_handler(static function (
2      int $severity,
3      string $message,
4      string $file,
5      int $line
6  ): bool {
7      if (!(error_reporting() & $severity)) {
8          return false;
9      }
10     throw new ErrorException($message, 0, $severity, $file, $line);
11 });
```

Слабый сигнал: catch (Exception) как гарантия перехвата TypeError; Пустой catch (Throwable) с продолжением обработки; Показ stack trace и внутренних сообщений в production HTTP-ответе.

10. Как выбирать pm-политику и параметры PHP-FPM на основании нагрузки, а не по шаблону из интернета?

Ответ: pm=static заранее держит фиксированное число workers и даёт предсказуемую задержку ценой постоянной памяти; dynamic меняет пул между min/max spare; ondemand экономит память редкого сервиса, но добавляет cold-start и хуже переносит всплеск. Во всех режимах один синхронный worker обслуживает один запрос одновременно.

Верхнюю границу оценивают по памяти и downstream capacity: доступная RAM делится не на средний, а на высокий перцентиль RSS worker с запасом для ОС и соседних процессов. Затем проверяют, что такое число соединений выдерживают БД и внешние сервисы. Little's Law связывает concurrency примерно с $\text{throughput} \times \text{latency}$, но длинный хвост требует нагрузочного теста.

Смотрят listen queue, active/idle workers, max children reached, p95/p99 latency, slowlog и saturation БД. Увеличение pm.max_children при забитой БД лишь создаёт очередь в другом месте; правильнее ограничить конкурентность, поставить deadlines и устранить медленный участок.

Слабый сигнал: Расчёт max_children только по общей RAM без измерения RSS worker; Безусловное увеличение числа workers при росте latency; Отсутствие slowlog, метрик очереди и лимитов downstream-соединений.

2. ООР, типизация и принципы проектирования

11. Как covariance и contravariance в PHP связаны с LSP и где разработчики чаще всего путают направление?

Ответ: Переопределяющий метод может сузить возвращаемый тип — covariance — и расширить тип параметра — contravariance. Клиент базового контракта по-прежнему может передать всё обещанное базой, а получить не менее конкретный результат. Обратные изменения нарушили бы подстановку и отвергаются движком.

Variance проверяет только сигнатуры, но LSP шире: наследник не должен усиливать предусловия, ослаблять постусловия или менять наблюдаемую семантику и исключения. Метод, типизированный корректно, всё равно нарушает LSP, если для части допустимых базовым контрактом значений начинает падать.

Инвариантные mutable-контейнеры особенно важны: Collection<Dog> нельзя считать Collection<Animal>, если через неё можно добавить Cat. PHPDoc generics анализируются статически Psalm/PHPStan, и variance-аннотация безопасна обычно лишь для read-only producer-интерфейса.

```
1 class Animal {}
2 class Dog extends Animal {}
3
4 interface Factory {
5     public function create(Dog $prototype): Animal;
6 }
7
8 final class DogFactory implements Factory {
9     public function create(Animal $prototype): Dog {
10         return new Dog();
11     }
12 }
```

PHP

Слабый сигнал: Попытка сузить тип параметра в наследнике; Сведение LSP только к совместимости PHP-сигнатур; Ковариантный mutable generic-контейнер.

12. Когда union, intersection и DNF-типы улучшают модель, а когда скрывают отсутствие нормальной абстракции?

Ответ: Union A|B означает альтернативу, intersection A&B — одновременное выполнение нескольких контрактов, а DNF позволяет объединять parenthesized intersections, например (Cacheable&Countable)|null. Эти типы полезны на адаптерных границах и при точном описании существующего протокола без искусственного общего предка.

Большой union из доменных классов часто превращает полиморфизм в цепочку instanceof, а mixed снимает почти все гарантии. Если набор вариантов закрыт и у каждого различается поведение, лучше общий интерфейс с методом; если различаются данные команды, статический анализ может моделировать discriminated union через PHPDoc shapes.

Тип не заменяет runtime validation: JSON, HTTP и DB возвращают недоверенные скаляры. На границе данные проверяют и превращают в типизированный DTO/value object, после чего внутренний код не таскает array<string,mixed>. never подходит для функций, которые гарантированно не возвращают управление, но злоупотреблять им ради подавления анализатора нельзя.

Слабый сигнал: Union, после которого в каждом потребителе идёт одинаковая цепочка instanceof; array или mixed как основной доменный контракт; Ожидание, что type hint валидирует семантику внешнего JSON.

13. Какие гарантии дают readonly properties и readonly class и почему это не глубокая неизменяемость?

Ответ: Readonly-свойство можно инициализировать один раз из разрешённой области, после чего нельзя присвоить заново, увеличить, unset или изменить элемент массива через это свойство. Readonly class распространяет правило на объявленные instance properties и запрещает динамические свойства, но не делает объектный граф immutable.

Если readonly-свойство хранит mutable-объект, его внутреннее состояние продолжает меняться. Для настоящего value object нужны неизменяемые зависимости, defensive copy на входе/выходе и wither, возвращающий новый экземпляр. Массивы фактически защищены от изменения через свойство, но переданные внутрь объекты — нет.

Неизменяемость упрощает конкурентное и кешируемое чтение, однако может повысить число аллокаций для больших графов. Entity с жизненным циклом не обязана быть readonly; важнее контролировать переходы состояния методами, чем маскировать публичными setters формальную изменяемость.

PHP

```

1  readonly final class Money {
2      public function __construct(
3          public int $minor,
4          public string $currency
5      ) {
6          if ($minor < 0) {
7              throw new InvalidArgumentException('Negative amount');
8          }
9      }
10
11     public function add(self $other): self {
12         if ($other->currency !== $this->currency) {
13             throw new LogicException('Currency mismatch');
14         }
15         return new self($this->minor + $other->minor, $this->currency);
16     }
17 }

```

Слабый сигнал: Утверждение, что readonly рекурсивно замораживает вложенные объекты; Публичный mutable DateTime внутри readonly value object; readonly как замена доменным инвариантам.

14. Чем late static binding отличается от self и где static:: создаёт хрупкие иерархии?

Ответ: self:: привязывается к классу, где метод объявлен, а static:: разрешается по runtime called class и поддерживает позднее статическое связывание. static как return type означает экземпляр вызванного класса и строже, чем self в fluent/factory API.

LSB полезен для named constructors и расширяемых фабрик, но static state разделяется и наследуется не всегда интуитивно, а private-методы нельзя безопасно считать полиморфными точками расширения. Вызов static::hook() фактически создаёт скрытый шаблонный метод: изменение сигнатуры или видимости в наследнике ломает базовый алгоритм.

Если вариативность является частью дизайна, её лучше выразить protected abstract методом, интерфейсом-фабрикой или внедрённой стратегией. final-класс с self проще, когда наследование не обещано; разрешать расширение «на всякий случай» — дорогое обязательство по обратной совместимости.

PHP

```

1  abstract class Record {
2      final public static function fromRow(array $row): static {
3          $object = new static();
4          $object->hydrate($row);
5          return $object;
6      }
7
8      abstract protected function hydrate(array $row): void;
9  }

```

Слабый сигнал: Смешивание self:: и static:: без понимания called class; LSB поверх глобального mutable static state; Публичное наследование класса без документированных extension points.

15. Как проектировать атрибуты и Reflection-based инфраструктуру без скрытой магии и проблем производительности?

Ответ: Attribute хранится в metadata; getAttributes() возвращает отражения, а экземпляр атрибута создаётся только при newInstance(), где уже может выполняться пользовательский конструктор. Фильтрация с IS_INSTANCEOF удобна для полиморфных атрибутов, но сама по себе не валидирует корректность места и комбинаций.

Reflection обычно не является главным bottleneck после OPcache, однако повторный обход всех классов и создание атрибутов на каждый запрос избыточны. Framework компилирует metadata в контейнер/route cache или кеширует нормализованный результат с ключом, зависящим от release; кеш нельзя слепо сериализовать между несовместимыми версиями кода.

Атрибут должен быть декларативным value object, без доступа к контейнеру, сети и БД. Ошибки конфигурации лучше обнаружить на этапе cache warmup, а не на первом production-запросе. Для ключевой бизнес-логики явный код читаемее, чем цепь обработчиков, обнаруживаемая только reflection.

Слабый сигнал: I/O или обращение к service locator из конструктора атрибута; Полное сканирование проекта reflection на каждый запрос; Атрибуты, порядок и конфликт которых не валидируются при сборке.

16. Как сохранить инварианты при clone, __serialize()/__unserialize() и восстановлении объектов?

Ответ: clone выполняет поверхностную копию свойств, после чего вызывает __clone() у нового объекта; вложенные объекты остаются общими, пока их явно не клонировать. Поэтому агрегат с mutable-коллекцией, токеном блокировки или identity не должен автоматически считаться независимой копией.

__serialize() задаёт устойчивое представление, а __unserialize() обязан проверить форму и восстановить инварианты. Десериализация недоверенных PHP-данных опасна: magic methods создают gadget chains, поэтому для внешнего контракта предпочтительны JSON/Protobuf плюс явный mapper; allowed_classes лишь снижает, но не устраняет риск.

Сериализованный объект не должен быть долговременной схемой домена без versioning и миграций. Часто безопаснее хранить primitives/event payload и восстанавливать объект через named constructor; ресурсы, closures и инфраструктурные проху в состояние не включают.

```
1  final class Period {
2      public function __construct(
3          private DateTimeImmutable $from,
4          private DateTimeImmutable $to
5      ) {
6          if ($from >= $to) {
7              throw new InvalidArgumentException('Invalid period');
8          }
9      }
10
11     public function __serialize(): array {
12         return ['v' => 1, 'from' => $this->from->format(DATE_ATOM), 'to' =>
13             $this->to->format(DATE_ATOM)];
14     }
15
16     public function __unserialize(array $data): void {
17         if (($data['v'] ?? null) !== 1) {
18             throw new UnexpectedValueException('Unsupported version');
19         }
20         $this->__construct(new DateTimeImmutable($data['from']), new
21             DateTimeImmutable($data['to']));
22     }
23 }
```

PHP

Слабый сигнал: unserialize() данных пользователя без строгого контроля типов; Ожидание глубокой копии от clone; Сохранение DI-контейнера, closure или DB connection в сериализуемом состоянии.

17. Где должна находиться composition root и почему Service Locator нарушает Dependency Inversion даже при наличии интерфейсов?

Ответ: Composition root — ограниченное место на краю приложения, где выбирают реализации, читают конфигурацию и строят object graph. В Symfony это compiled container/bootstrap, в Laravel — service providers/container bindings; доменные классы получают уже готовые зависимости через конструктор.

Service Locator скрывает реальные зависимости за container->get(), переносит ошибки на runtime и позволяет любому классу запросить что угодно. Наличие интерфейса у полученного сервиса не исправляет контракт: сигнатура конструктора всё ещё ложно говорит, что объект самодостаточен.

Иногда locator-подобный registry оправдан для plugin runtime с динамическим ключом, но его заворачивают в узкий типизированный интерфейс, например HandlerResolver. Не надо инъектировать десять реализаций «на будущее»: composition root может собрать стратегию или map только для поддерживаемых случаев.

Слабый сигнал: Передача ContainerInterface в application/domain service; Статический глобальный facade как скрытый locator в бизнес-логике; Тесты, которые вынуждены конфигурировать весь контейнер для одного класса.

18. Как провести границу агрегата и обеспечить его инварианты при конкурентных изменениях?

Ответ: Агрегат — граница согласованности: одна root entity контролирует изменения внутренних объектов, а внешний код ссылается на другие агрегаты по identity. Не стоит включать весь связанный граф; большой агрегат увеличивает contention, объём загрузки и вероятность конфликтов.

Инвариант внутри одного агрегата проверяется его методом и фиксируется одной транзакцией. Межагрегатное правило обычно достигается процессом, reservation или eventual consistency, а не распределённой блокировкой. Репозиторий загружает и сохраняет root, не выдавая наружу mutable внутренности.

Проверка в памяти не защищает от двух параллельных команд. Нужны optimistic version column с compare-and-swap либо осознанная pessimistic lock; unique/check constraints остаются последней гарантией. Конфликт версии — отдельный ожидаемый исход, который нельзя бездумно бесконечно retry.

```

1  UPDATE account
2  SET balance = :balance, version = version + 1
3  WHERE id = :id AND version = :expected_version;
4
5  -- rowCount() === 0 означает конкурентный конфликт,
6  -- а не «запись не требовалась»

```

SQL

Слабый сигнал: Агрегат, содержащий почти всю предметную область; Проверка уникальности только SELECT-запросом без DB constraint; Last-write-wins для финансового или квотного инварианта.

19. Когда выбирать Decorator, middleware pipeline или event subscribers для cross-cutting поведения?

Ответ: Decorator сохраняет тот же узкий контракт и явно оборачивает одну зависимость: кеш, retry, метрики или авторизация видны в object graph. Он удобен, когда порядок фиксирован и поведение относится к конкретному порту, например PriceProvider.

Middleware моделирует цепочку request/command → next → response; оно подходит для однородного pipeline и short-circuit, но порядок становится частью семантики. Transaction middleware снаружи retry middleware ведёт себя иначе, чем внутри; порядок следует тестировать как отдельный контракт.

Events уменьшают связанность publisher с дополнительными реакциями, но синхронный subscriber всё ещё увеличивает latency и может сорвать исходную транзакцию. Критический результат не стоит прятать в событии; для асинхронной доставки нужны durable message/outbox, идемпотентность и наблюдаемость.

Слабый сигнал: Случайный порядок middleware из порядка сканирования файлов; Критическая бизнес-операция только в необязательном event listener; Decorator, меняющий смысл или тип результата базового контракта.

20. Как проектировать PHP API с учётом реальной обратной совместимости, включая named arguments и наследование?

Ответ: Публичный контракт включает не только имена классов и типы: parameter names становятся API из-за named arguments, порядок и default values влияют на вызовы, исключения и побочные эффекты — на поведение. Добавление метода в широко реализуемый interface ломает реализации, а добавление abstract method — наследников.

Расширяемость лучше обещать явно: маленькие интерфейсы, final по умолчанию для классов без extension contract, @internal для нестабильных точек и factories вместо публичных сложных конструкторов. Protected surface — тоже контракт; переименование protected method может быть BC break.

SemVer не освобождает от проверки: API diff-инструмент, статический анализ downstream-проектов и deprecation period до major-релиза снижают риск. Новую опцию чаще безопаснее добавить в immutable options object или новый overload-like named constructor, чем растить список скалярных параметров.

Слабый сигнал: Переименование параметра публичного метода как «внутренний рефакторинг»; Добавление метода в public interface в minor-релизе без стратегии совместимости; Неограниченное наследование concrete-классов без документированного контракта.

3. Composer, стандарты и внутреннее устройство framework-подходов

21. Как Composer разрешает зависимости и почему composer update в production — архитектурная ошибка процесса доставки?

Ответ: Composer строит SAT-задачу из constraints корневого пакета, транзитивных требований, conflicts/replaces/provides, версии PHP и extension platform. Несовместимость может быть вызвана не ближайшим пакетом, а набором решений; composer why-not и prohibits помогают получить цепочку ограничений.

composer update выбирает новые версии и переписывает composer.lock, а composer install при наличии lock воспроизводит уже выбранный граф. Поэтому update выполняют в контролируемой среде, review-ят diff lock-файла и тестируют артефакт; production получает install --no-dev по закоммиченному lock, часто внутри image build.

config.platform помогает унифицировать solver с production, но может скрыть отсутствие реального расширения, поэтому composer check-platform-reqs запускают в целевой среде. Частичный update способен изменить связанные транзитивные пакеты; флаг -W применяют осознанно и анализируют весь lock diff.

Слабый сигнал: composer update во время production-депоя; Игнорирование composer.lock в приложении; config.platform без проверки фактической production-платформы.

22. Как работают PSR-4, classmap-authoritative и APCu autoloader и какие классы они могут «потерять»?

Ответ: PSR-4 преобразует namespace prefix в один или несколько каталогов и проверяет путь только при первом обращении к неизвестному классу. Optimized autoloader строит classmap известных классов; --classmap-authoritative дополнительно запрещает fallback-поиск для отсутствующих в тар имён.

Authoritative mode ускоряет miss, но ломает runtime-generated классы и классы, появившиеся после dump-autoload, если генератор не включён в build. APCu mode кеширует positive/negative lookups в shared memory соответствующего PHP/FPM-инстанса; release prefix должен меняться, иначе отрицательный кеш способен пережить появление класса в новом релизе.

Файлы, подключаемые через autoload.files, исполняются при загрузке vendor/autoload.php и создают глобальные side effects — их стоит минимизировать. В production autoload генерируют после code generation, а корректность проверяют отдельным smoke test на чистом release.

```
1  {
2      "autoload": {
3          "psr-4": {
4              "App\\": "src/"
5          },
6          "exclude-from-classmap": [
7              "/Tests/"
8          ]
9      }
10 }
```

JSON

Слабый сигнал: Генерация проху после composer dump-autoload -a; Одинаковый APCu prefix для несовместимых релизов; Бизнес-инициализация через autoload.files.

23. Как уменьшить supply-chain risk Composer plugins, scripts и транзитивных пакетов?

Ответ: Composer plugin исполняет PHP-код внутри самого Composer и имеет возможности текущего пользователя; scripts также запускают команды на install/update. allow-plugins должен быть явным allowlist, а CI для недоверенного pull request не должен автоматически выполнять новые plugins/scripts с production credentials.

Lock-файл фиксирует версии и dist references, но не доказывает безопасность пакета. Нужны composer audit, review смены maintainer/package source, минимизация зависимостей, приватный mirror или provenance-политика для критичной среды и оперативное обновление уязвимых пакетов.

Build делают непривилегированным, отделяют секреты от dependency step и по возможности создают неизменяемый vendor-артефакт. --no-scripts полезен для inspection, но если приложению реально нужны generation scripts, их запускают отдельным разрешённым шагом и проверяют получившийся diff.

```
1  {
2      "config": {
3          "allow-plugins": {
4              "symfony/flex": true,
5              "**": false
6          }
7      }
8  }
```

JSON

Слабый сигнал: allow-plugins=true для всех пакетов; Dependency install с облачными production-ключами в окружении; Автоматическое принятие большого composer.lock diff без review.

24. Какие тонкости SemVer и Composer constraints важны владельцу публичной PHP-библиотеки?

Ответ: Constraint ^2.3 означает >=2.3.0 <3.0.0, но для версий до 1.0 caret уже: ^0.3 не допускает 0.4. Wildcard и диапазоны объединяются по правилам Composer, а minimum-stability и prefer-stable влияют на выбор pre-release; слишком широкое * перекладывает риск на потребителя.

SemVer описывает обещание совместимости, но PHP BC surface шире сигнатур: parameter names, traits, сериализованная форма, protected members, исключения и даже порядок side effects могут использоваться downstream. Публичные constants и enum cases тоже нельзя бездумно менять в minor.

Библиотека обычно требует диапазон поддерживаемых версий зависимостей, а не фиксирует их lock для потребителя; lock полезен для тестирования собственного проекта, но не навязывает транзитивные версии установщику. Матрица CI должна проверять нижнюю и верхнюю поддерживаемые комбинации, иначе заявленный диапазон фиктивен.

Слабый сигнал: Зависимость "*" в публичной библиотеке; Гарантия совместимости только потому, что PHP не выдаёт fatal error; Тестирование только на самой новой версии каждой зависимости.

25. Какие семантические ловушки PSR-7 и PSR-15 возникают в middleware stack?

Ответ: PSR-7 message immutable на уровне API: withHeader()/withBody() возвращают новый message, и игнорирование результата теряет изменение. При этом StreamInterface оборачивает mutable cursor/resource, поэтому две копии response могут разделять поток; чтение body меняет позицию и требует осознанного rewind.

PSR-15 middleware решает, вызвать ли следующий handler, и может short-circuit ответом. Код до handle() идёт по порядку регистрации, после — в обратном, поэтому error handling, transactions, routing, authentication и response compression нельзя переставлять без анализа.

Большие body не следует безусловно преобразовывать в string: это разрушает streaming и может исчерпать память. Middleware должно сохранять контракт атрибутов request, корректно закрывать/передавать потоки и не помещать request-scoped состояние в singleton-свойства долгоживущего worker.

```

1  final class CorrelationMiddleware implements MiddlewareInterface {
2      public function process(
3          ServerRequestInterface $request,
4          RequestHandlerInterface $handler
5      ): ResponseInterface {
6          $id = $request->getHeaderLine('X-Correlation-ID') ?: bin2hex(random_bytes(8));
7          $request = $request->withAttribute('correlation_id', $id);
8
9          return $handler->handle($request)
10             ->withHeader('X-Correlation-ID', $id);
11     }
12 }

```

Слабый сигнал: Вызов `withHeader()` без присваивания возвращённого объекта; Приведение любого `request/response` body к строке; Request-specific данные в свойстве singleton middleware.

26. Что происходит внутри compiled DI container и какие проблемы проявляются в long-running PHP workers?

Ответ: Container собирает definitions, aliases, factories и compiler passes, разрешает autowiring по типам, затем часто генерирует оптимизированный PHP-код. Compile-time validation переносит circular/missing dependency ошибки на build; runtime reflection-контейнер проще, но обнаруживает часть проблем только под нагрузкой.

Shared service обычно singleton в пределах container instance, а не глобально навсегда. В классическом FPM container создаётся или bootstraps на запрос и request state исчезает; в RoadRunner, Swoole или queue worker тот же объектный граф может обслуживать множество задач, поэтому mutable request state протекает между ними.

Нужны явные scopes/resettable services, очистка EntityManager, лог-контекста и кешей после каждой задачи. Lazy proxy сокращает startup лишь пока зависимость не используется и может отложить ошибку конфигурации; proxy также усложняет final/readonly-классы и stack traces.

Слабый сигнал: Ожидание, что singleton автоматически безопасен между запросами long-running worker; Autowiring нескольких реализаций без явного binding; Lazy services как способ скрыть циклическую зависимость.

27. Как работают Identity Map и Unit of Work в ORM и почему они вызывают N+1, stale data и рост памяти?

Ответ: Identity Map гарантирует один object instance на пару type+id внутри EntityManager/session. Unit of Work хранит snapshots, вычисляет change sets и упорядочивает INSERT/UPDATE/DELETE при flush; flush не обязан означать commit, если внешняя транзакция ещё открыта.

Lazy proxy/collection выполняет запрос при первом доступе, поэтому цикл по сущностям легко создаёт N+1. Решение выбирают по use case: fetch join, batch loading, projection/read model; бездумный join нескольких to-many коллекций создаёт декартово произведение и тяжёлую гидрацию.

Долгий worker накапливает managed entities и может отдавать устаревшее состояние из Identity Map. Батч должен flush/clear по границе, но после clear объекты detached и изменения уже не отслеживаются. Транзакции и DB constraints проектируют явно — ORM не превращает несколько flush в атомарный бизнес-процесс.

```

1  foreach ($commands as $i => $command) {
2      $handler($command);
3
4      if (($i + 1) % 200 === 0) {
5          $entityManager->flush();
6          $entityManager->clear(); // прежние entity теперь detached
7      }
8  }
9
10 $entityManager->flush();
11 $entityManager->clear();

```

Слабый сигнал: Доступ к lazy relation внутри большого цикла без query-count теста; Один EntityManager на бесконечный worker без clear/reset; Fetch join нескольких больших to-many коллекций как универсальное лечение.

28. Как устроен lifecycle HTTP-запроса в Symfony/Laravel и где размещать транзакции, обработку исключений и after-response работу?

Ответ: Оба подхода превращают server globals в Request, прогоняют kernel/middleware pipeline, выполняют routing/controller и нормализуют Response. Symfony широко использует kernel events/subscribers, Laravel — middleware, service container и exception handler; детали различаются, но границы ответственности одинаковы.

Транзакция должна охватывать конкретный application use case, а не весь HTTP kernel: иначе сериализация ответа, внешние вызовы и streaming держат блокировки. Exception mapper на внешней границе переводит доменные и инфраструктурные ошибки в стабильные HTTP-коды, но не должен повторно выполнять неидемпотентный controller.

Terminable middleware/kernel.terminate запускается после формирования или отправки response, но в обычном FPM всё ещё занимает worker и не гарантирует durable execution при падении процесса. Критичную работу передают в очередь через transactionally recorded outbox; after-response оставляют для best-effort телеметрии.

Слабый сигнал: DB transaction, открытая на всю длительность HTTP-запроса; Критичная отправка письма только в kernel.terminate/terminate middleware; Контроллер, содержащий одновременно transport mapping, бизнес-правила и persistence.

29. Как организовать конфигурацию и cache warmup Symfony/Laravel для immutable deployment?

Ответ: Конфигурация должна быть явным входом приложения: обязательные значения валидируются при boot, переводятся в типизированные config-объекты и не читаются через env() глубоко в domain-коде. Секреты, deploy-time параметры и динамические feature flags имеют разный жизненный цикл; смешивание их в одном глобальном массиве делает поведение нерепродуцируемым.

Compiled container, routes, annotations/attributes metadata и optimized Composer autoloader прогревают до включения release в traffic. Warmup выполняют для того же PHP, extensions и набора build-time параметров, что будут в runtime; общий writable cache между двумя версиями опасен несовместимыми ключами и частично очищенным состоянием.

Release является неизменяемым artifact с version/checksum. Переключение каталога или image tag атомарно, затем контролируемо сбрасывают или перезапускают OPcache/workers. Runtime-секреты не запекают в image, но их наличие проверяют readiness; ручное редактирование production-конфига заменяют версионизуемым изменением и аудитом.

```

1  FROM composer:2 AS composer
2  FROM php:8.5-fpm-alpine AS runtime
3  COPY --from=composer /usr/bin/composer /usr/bin/composer
4  WORKDIR /app
5  COPY composer.json composer.lock ./
6  RUN composer install --no-dev --no-scripts --no-autoloader
7  COPY . .
8  RUN composer dump-autoload --no-dev --classmap-authoritative \
9      && APP_ENV=prod php bin/console cache:warmup
10 USER www-data
11 CMD ["php-fpm", "-F"]

```

DOCKERFILE

Слабый сигнал: composer update или cache clear при старте каждого production pod; Общий writable framework cache для одновременно работающих release; Чтение env() в произвольных domain-классах; Секреты, навсегда запечённые в container image.

30. Как запускать scheduler/cron в нескольких экземплярах приложения без дублей и потерянных задач?

Ответ: Флаг withoutOverlapping или один leader уменьшает конкуренцию, но не превращает запуск в exactly-once: процесс может выполнить side effect и умереть до фиксации статуса. Надёжная единица — идемпотентная job с детерминированным ключом периода, например report:tenant:2026-07-25, и unique constraint в хранилище запусков.

Scheduler должен только обнаружить due work и в одной DB-транзакции записать запуск вместе с outbox-событием. Отдельный dispatcher публикует outbox в durable queue; это закрывает окно между фиксацией claim и отправкой сообщения. Lease или leader election могут уменьшать конкуренцию, но handler всё равно обязан выдерживать повторную доставку.

Нужны метрики schedule delay, duration, success/failure, skipped duplicate и age последнего успешного запуска. Backfill запускают тем же протоколом с явным диапазоном, ограничением параллелизма и checkpoint; timezone/DST исключают хранением расписания и ключей периода в UTC либо заранее определённой бизнес-зоне.

```

1  BEGIN;
2  WITH claimed AS (
3      INSERT INTO scheduled_run (job_name, period_key, status)
4      VALUES ('daily-invoice', :period_key, 'scheduled')
5      ON CONFLICT (job_name, period_key) DO NOTHING
6      RETURNING id
7  )
8  INSERT INTO outbox (event_id, topic, payload)
9  SELECT :event_id, 'scheduled.daily-invoice', :payload
10 FROM claimed;
11 COMMIT;
12
13 -- Dispatcher публикует outbox; повторный scheduler
14 -- не создаст ни второй запуск, ни второе событие.

```

Слабый сигнал: Надежда на единственный pod scheduler без контроля его живости; Lock без lease, owner token и процедуры takeover; Прямая публикация в очередь после commit claim без transactional outbox; Неидемпотентная задача, считающая отсутствие параллельного старта гарантией; Локальное время сервера как неявный ключ периода.

4. Базы данных, SQL, транзакции и ORM

31. Как работают уровни изоляции транзакций и какие аномалии они предотвращают?

Ответ: Изоляция определяет, какие изменения параллельных транзакций видит текущая транзакция. Read Committed обычно дает новый снимок на каждый statement и допускает non-repeatable read; Repeatable Read фиксирует снимок на транзакцию, но конкретная защита от phantom и write skew зависит от СУБД. Например, PostgreSQL Repeatable Read основан на snapshot isolation, а InnoDB дополнительно применяет next-key locks для locking reads.

Serializable должен давать эффект некоторого последовательного выполнения, но достигается либо блокировками, либо обнаружением опасных зависимостей. Поэтому приложение обязано уметь повторять всю транзакцию после serialization failure или deadlock: нельзя ретраить только последний UPDATE, если решения принимались по ранее прочитанным данным.

Уровень выбирают по инварианту, а не по привычке. Для независимого обновления строки часто достаточно Read Committed плюс атомарный UPDATE; для проверки лимита по нескольким строкам нужны явные блокировки, ограничение в схеме или Serializable. Более строгая изоляция уменьшает класс гонок, но увеличивает конфликты, ожидания и число ретраев.

```

1  BEGIN ISOLATION LEVEL SERIALIZABLE;
2  SELECT COALESCE(SUM(amount), 0) AS reserved
3  FROM reservations
4  WHERE account_id = 42;
5  INSERT INTO reservations(account_id, amount) VALUES (42, 100);
6  COMMIT; -- при SQLSTATE 40001 повторить транзакцию целиком

```

Слабый сигнал: Утверждение, что Repeatable Read одинаков во всех СУБД; Надежда на Serializable без обработки serialization failure; Использование более строгой изоляции вместо формулировки инварианта.

32. Почему возникают дедлоки и как их диагностировать и предотвращать?

Ответ: Дедлок возникает, когда транзакции образуют цикл ожидания: T1 держит ресурс A и ждет B, а T2 держит B и ждет A. СУБД обнаруживает цикл в wait-for graph и откатывает одну транзакцию; это штатный механизм разрешения конкуренции, а не обязательно дефект самой базы.

Для диагностики сохраняют текст запросов, параметры, длительность транзакции и deadlock graph из журнала СУБД. Важно найти реальные заблокированные строки и индексы: запрос без подходящего индекса может захватить намного больше записей, чем предполагает разработчик, а внешний HTTP-вызов внутри транзакции растягивает окно конфликта.

Основные меры: одинаковый порядок захвата ресурсов, короткие транзакции, подходящие индексы, минимальный набор блокировок и повтор транзакции с ограниченным backoff. Ретраи не заменяют исправление систематического дедлока и должны быть безопасны относительно внешних побочных эффектов.

```

1  BEGIN;
2  SELECT id, balance
3  FROM accounts
4  WHERE id IN (10, 20)
5  ORDER BY id
6  FOR UPDATE;
7  -- все переводы блокируют счета в одном порядке
8  UPDATE accounts SET balance = balance - 50 WHERE id = 10;
9  UPDATE accounts SET balance = balance + 50 WHERE id = 20;
10 COMMIT;
```

SQL

Слабый сигнал: Транзакции с сетевыми вызовами или пользовательским вводом; Разный порядок обновления одних и тех же сущностей; Бесконечный немедленный retry без лимита и jitter.

33. Как проектировать составные индексы и понимать, действительно ли они помогают?

Ответ: B-tree индекс упорядочен по полному ключу, поэтому важен принцип левого префикса. Обычно сначала ставят колонки с проверкой на равенство, затем диапазон и поля сортировки, но механическое правило «самая селективная первой» недостаточно: порядок должен соответствовать реальным предикатам, ORDER BY и распределению данных.

Covering index позволяет выполнить запрос без чтения основной таблицы, однако расширяет индекс и повышает стоимость INSERT, UPDATE, VACUUM и хранения. Низкоселективный флаг сам по себе редко полезен, но частичный индекс по редкому состоянию может быть очень эффективен. Функция над индексируемой колонкой обычно ломает SARGability, если нет функционального индекса.

Решение подтверждают на репрезентативных данных через EXPLAIN ANALYZE, статистику buffer reads и p95/p99, а не по факту появления Index Scan. Неиспользуемые и дублирующие индексы удаляют только после проверки статистики всех типов нагрузки, включая фоновые задания и редкие административные запросы.

```

1  -- PostgreSQL: равенство, затем диапазон; INCLUDE делает индекс покрывающим
2  CREATE INDEX CONCURRENTLY idx_orders_customer_created
3  ON orders (customer_id, created_at DESC)
4  INCLUDE (status, total)
5  WHERE deleted_at IS NULL;
6
7  SELECT created_at, status, total
8  FROM orders
9  WHERE customer_id = 42 AND deleted_at IS NULL
10     AND created_at < TIMESTAMP '2026-07-01'
11  ORDER BY created_at DESC
12  LIMIT 50;
```

SQL

Слабый сигнал: Индекс на каждую колонку без учета формы запросов; Оценка индекса только по селективности первого поля; Игнорирование стоимости записи и размера working set.

34. Как системно оптимизировать медленный SQL-запрос?

Ответ: Сначала фиксируют конкретный запрос, параметры, частоту, p95 и объем возвращаемых данных. Затем читают фактический план: estimated rows против actual rows, типы join, число циклов, сортировки на диске, buffer hits/reads и время по узлам. Большая ошибка оценки часто указывает на устаревшую статистику, коррелированные колонки или сильный data skew.

После этого уменьшают работу: делают условия SARGable, отбрасывают строки раньше, устраняют лишние JOIN и SELECT *, добавляют подходящий индекс или меняют модель доступа. Переписывание CTE, EXISTS вместо размножающего JOIN и предварительная агрегация полезны только когда план и измерения показывают причину.

EXPLAIN ANALYZE реально выполняет запрос, поэтому для изменяющего SQL его запускают в откатываемой транзакции или на безопасной копии. Финальную версию проверяют на разных параметрах и объемах: хороший план для одного клиента может стать катастрофическим для tenant с миллионами строк.

```
1  -- Несаргируемо: индекс по created_at обычно не помогает
2  WHERE DATE(created_at) = DATE '2026-07-25'
3
4  -- Саргируемый полуинтервал
5  WHERE created_at >= TIMESTAMP '2026-07-25 00:00:00'
6        AND created_at <  TIMESTAMP '2026-07-26 00:00:00'
7
8  EXPLAIN (ANALYZE, BUFFERS, WAL, FORMAT TEXT)
9  SELECT id FROM orders
10 WHERE created_at >= TIMESTAMP '2026-07-25 00:00:00'
11        AND created_at <  TIMESTAMP '2026-07-26 00:00:00';
```

SQL

Слабый сигнал: Оптимизация без actual plan и production-подобных данных; Слепое добавление индекса после любого Seq Scan; Игнорирование количества round-trip и объема результата.

35. Чем keyset pagination лучше OFFSET и какие у нее ограничения?

Ответ: OFFSET заставляет СУБД найти и отбросить предыдущие строки, поэтому стоимость растет с номером страницы. При параллельных вставках и удалениях границы смещаются: пользователь может увидеть дубликаты или пропуски. Keyset pagination продолжает чтение после последнего ключа и имеет стоимость, близкую к постоянной при подходящем индексе.

Порядок обязан быть детерминированным, поэтому к неуникальному created_at добавляют уникальный tie-breaker, например id. Поля курсора должны быть NOT NULL; иначе заранее определяют NULLS FIRST/LAST и отдельную ветку cursor predicate, потому что сравнение кортежа с NULL даст UNKNOWN. Cursor кодирует значения непрозрачно и подписывается, если клиенту нельзя доверять.

Keyset не дает дешевого перехода на произвольную страницу и усложняет двунаправленную навигацию. Если бизнесу нужен «перейти на страницу 417» или точное общее число, полезны отдельный поисковый индекс, периодические счетчики либо ограниченный OFFSET для небольших наборов.

```
1  CREATE INDEX idx_feed_created_id
2  ON feed_items (created_at DESC, id DESC);
3  -- Предусловие: feed_items.created_at IS NOT NULL.
4
5  SELECT id, created_at, title
6  FROM feed_items
7  WHERE (created_at, id) < (:cursor_created_at, :cursor_id)
8  ORDER BY created_at DESC, id DESC
9  LIMIT 50;
```

SQL

Слабый сигнал: Keyset по неуникальному полю без tie-breaker; Курсор, содержащий изменяемый номер страницы; Обещание произвольного перехода по страницам без дополнительного механизма.

36. Как обеспечить корректность конкурентной записи без схемы «сначала SELECT, потом INSERT»?

Ответ: Проверка существования и последующая вставка — две операции, между которыми другая транзакция может вставить ту же сущность. Инвариант должен закрепляться UNIQUE, FOREIGN KEY, CHECK или exclusion constraint, а приложение должно интерпретировать конкретное нарушение ограничения как ожидаемый исход.

Для create-or-update используют атомарный UPSERT с ясно определенной семантикой конфликта. Важно не превращать любой конфликт в успех: совпадение idempotency key с другим payload означает ошибку, а безусловный UPDATE может потерять новое значение при last-write-wins.

Ограничения базы — последняя линия защиты, но они не отменяют доменную валидацию и понятные ошибки API. При высокой конкуренции также оценивают hotspot уникального индекса и длительность блокировки конфликтующей строки.

```
SQL
1 CREATE UNIQUE INDEX uq_payment_request
2 ON payments (merchant_id, idempotency_key);
3
4 INSERT INTO payments(merchant_id, idempotency_key, payload_hash, status)
5 VALUES (:merchant, :key, :hash, 'pending')
6 ON CONFLICT (merchant_id, idempotency_key) DO NOTHING
7 RETURNING id, payload_hash, status;
8
9 -- Если RETURNING пуст, прочитать существующую строку
10 -- и проверить, что payload_hash совпадает.
```

Слабый сигнал: SELECT exists перед INSERT без уникального ограничения; Перехват всех SQL-ошибок как дубликата; UPSERT с неявной last-write-wins семантикой.

37. Как учитывать replication lag при чтении с реплик?

Ответ: Асинхронная реплика подтверждает запись позже primary, поэтому сразу после успешного COMMIT чтение с replica может вернуть старое состояние или 404. Это не «редкий баг кеша», а ожидаемая модель eventual consistency; lag имеет распределение и резко растет при сетевых проблемах, длинных транзакциях или тяжелом replay.

Для read-your-writes применяют sticky primary на короткое окно, маршрутизацию критичных чтений на primary либо causal token: после записи клиент получает LSN/GTID, а чтение идет на реплику только когда она догнала этот маркер. Простая задержка sleep ненадежна и ухудшает latency.

Отдельно классифицируют запросы: каталог может терпеть секунды устаревания, а подтверждение платежа или авторизация — нет. Нужны метрики lag в байтах и времени, health-check реплик, ограничение максимальной допустимой отсталости и автоматический fallback, который не перегрузит primary при массовой деградации.

Слабый сигнал: Ожидание read-after-write от асинхронной реплики по умолчанию; Фиксированный sleep как гарантия консистентности; Мгновенный перевод всего чтения на primary без load-shedding.

38. Когда нужны partitioning и sharding и чем они принципиально отличаются?

Ответ: Partitioning делит таблицу внутри одной логической СУБД. Он полезен для pruning по времени или tenant, быстрого удаления старых секций и управления индексами, но сам по себе не увеличивает пределы CPU, памяти и записи одного сервера. Запрос без partition key способен пройти по всем секциям и стать медленнее.

Sharding распределяет данные по независимым узлам и добавляет новый класс задач: выбор shard key, маршрутизацию, ребалансировку, global uniqueness, fan-out запросы, cross-shard JOIN и транзакции. Хороший ключ равномерно распределяет нагрузку и совпадает с главной границей доступа; монотонный timestamp или один крупный tenant создают hotspot.

До sharding обычно исчерпывают вертикальное масштабирование, индексы, архивирование, replicas и partitioning. Если sharding неизбежен, закладывают versioned routing, виртуальные бакеты, observability по shard и процедуру онлайн-переноса с двойным чтением или верификацией, а не жесткий modulo по числу серверов.

Слабый сигнал: Ожидание, что partitioning автоматически распределит нагрузку по серверам; Shard key, не присутствующий в основных запросах; Хеширование по текущему числу узлов без стратегии ребалансировки.

39. Как обрабатывать миллионы строк без роста памяти, OFFSET-деградации и огромной транзакции?

Ответ: Для конечного batch-job по immutable или временно quiesced источнику в начале фиксируют high-water mark, например MAX(id), и читают по PK: id > checkpoint AND id <= high_watermark. На живой таблице sequence может выдать меньший id транзакции, которая commit-ится уже после прохода checkpoint, поэтому одного MAX(id) недостаточно: используют claim по status/processed_at через SKIP LOCKED, CDC commit-position либо overlap с reconciliation scan.

Каждая порция имеет ограниченный размер, собственную короткую транзакцию и checkpoint последнего подтверждённого id. Handler делает chunk идемпотентным, потому что падение между commit и сохранением checkpoint повторит часть диапазона. Один snapshot на часы даёт bloat, replication lag и удерживает старые версии строк.

Для параллельных workers сначала выполняют SELECT ... FOR UPDATE SKIP LOCKED, а затем UPDATE выбранных строк в той же транзакции, либо используют эквивалентный CTE. Учитывают starvation и повтор зависших lease. Измеряют rows/s, время транзакции, lock wait, WAL и lag; batch size подбирают по этим сигналам.

```
1  -- Только для immutable/quiesced source:
2  -- SELECT MAX(id) AS high_watermark
3  SELECT id, payload
4  FROM audit_event
5  WHERE id > :last_id
6        AND id <= :high_watermark
7  ORDER BY id
8  LIMIT 1000;
9
10 -- После commit сохраняем последний id как checkpoint.
11 -- Повтор последней порции безопасен благодаря idempotency key.
```

SQL

Слабый сигнал: Пагинация большого batch через OFFSET; Одна транзакция на весь многомиллионный набор; Передача полных ORM entity в долгоживущий worker; Checkpoint до commit обработанной порции; MAX(id) на live source без overlap, claim или reconciliation.

40. Как выполнять zero-downtime миграции схемы?

Ответ: Безопасная схема — expand/contract. Сначала добавляют обратно совместимую структуру, затем выкатывают код, который умеет работать с обеими версиями, выполняют backfill небольшими идемпотентными порциями и переключают чтение. Старую колонку или таблицу удаляют отдельным релизом после проверки, что старых экземпляров приложения больше нет.

DDL может удерживать тяжелые locks или переписывать всю таблицу, поэтому поведение проверяют для конкретной версии СУБД. В PostgreSQL CREATE INDEX CONCURRENTLY нельзя выполнять внутри transaction block: такой шаг делают нетранзакционной миграцией, а после сбоя проверяют и удаляют invalid index перед повтором. Перед уникальным преобразованием ищут существующие конфликты и определяют правило их разрешения.

До backfill новый код dual-write-ит normalized_email для новых и изменяемых строк; затем старые строки обновляют идемпотентными порциями. Lock timeout делают коротким, а миграцию — наблюдаемой и возобновляемой. Dual write удаляют после сверки, переключения чтения и завершения rollback-window.

Dual write не должен становиться постоянным состоянием: определяют источник истины, метрики расхождений и срок удаления совместимости. Для критичных преобразований запускают shadow read или сверку агрегатов до переключения, а rollback проектируют заранее — иногда откат кода возможен только пока старая колонка еще поддерживается.

SQL

```

1  -- Релиз 1: расширение без тяжелого DEFAULT
2  ALTER TABLE users ADD COLUMN normalized_email text;
3  SELECT lower(email), count(*) FROM users
4  GROUP BY lower(email) HAVING count(*) > 1;
5  -- Конфликты разрешаются до backfill; новый код dual-write-ит поле.
6  CREATE UNIQUE INDEX CONCURRENTLY uq_users_normalized_email
7  ON users (normalized_email)
8  WHERE normalized_email IS NOT NULL;
9
10 -- Backfill выполняется повторяемыми порциями по PK
11 UPDATE users
12 SET normalized_email = lower(email)
13 WHERE id > :last_id AND id <= :next_id
14     AND normalized_email IS NULL;
15
16 -- NOT NULL и удаление старого поля — только после переключения кода.

```

Слабый сигнал: Переименование или удаление поля в том же релизе, где меняется код; Backfill всей таблицы одной транзакцией; DDL без оценки блокировок и влияния на репликацию.

5. Архитектура, DDD, модульный монолит и CQRS

41. Что дает гексагональная архитектура и где должна проходить граница зависимостей?

Ответ: Гексагональная архитектура отделяет бизнес-правила от способов доставки и хранения. Application layer реализует use cases, domain содержит инварианты, а HTTP, CLI, Doctrine, Kafka и внешние SDK являются адаптерами. Зависимости направлены внутрь: доменная модель не импортирует фреймворк или инфраструктурный клиент.

Порты проектируются от потребностей use case, а не копируют API поставщика. Например, `PaymentGateway::authorize(Money, PaymentId)` стабильнее, чем утечка Stripe SDK во все слои. Адаптер переводит доменные типы, ошибки и idempotency semantics в протокол конкретной системы.

Цена подхода — больше явных интерфейсов и mapping-кода, поэтому не нужен интерфейс для каждого класса. Граница оправдана там, где есть изменчивость, внешнее IO или важное правило; простая чистая функция может оставаться без церемоний. Архитектура проверяется направлением импортов и тестами, а не названием каталогов.

Слабый сигнал: Domain entity с зависимостью от ORM, HTTP request или SDK; Интерфейсы один-в-один для каждого класса без границы; Порт, полностью повторяющий нестабильный API поставщика.

42. Как определять bounded context и строить context map?

Ответ: Bounded context — граница, внутри которой термины ubiquitous language имеют однозначный смысл, модель и владельца. Границу ищут по различающимся правилам, жизненному циклу данных, частоте совместных изменений и необходимости независимого релиза, а не по таблицам или слоям. Слово Customer, например, может означать плательщика в Billing и получателя в Delivery.

Context map фиксирует отношения и направление влияния: Partnership, Customer/Supplier, Conformist, Shared Kernel и Anti-Corruption Layer. Для каждой связи указывают владельца контракта, способ интеграции, SLA, версионирование и допустимую согласованность. Это делает организационную зависимость видимой до того, как она превратится в циклический импорт модулей.

Границы сверяют с потоками изменений и ответственностью команд: если две части постоянно меняются одной командой и обязаны деплоиться вместе, искусственное разделение дорого. Если модель, темп изменений и требования к доступности различаются, отдельный context полезен даже внутри модульного монолита. Context map пересматривают вместе с архитектурными и командными изменениями.

TEXT

```

1 Checkout (Customer) --> Pricing (Supplier)
2   contract: Quote.v2; owner: Pricing
3
4 Billing --> Legacy ERP
5   relation: Anti-Corruption Layer
6
7 Delivery --> Address Registry
8   relation: Conformist; risk accepted

```

Слабый сигнал: Одна каноническая модель данных для всей компании; Граница context, совпавшая со слоем Controller/Service/Repository; Shared Kernel без совместного владения и строгой политики изменений; Context map без направления зависимости и владельца контракта.

43. Чем Value Object отличается от Entity и зачем это важно в PHP?

Ответ: Entity определяется устойчивой идентичностью и жизненным циклом: Customer остаётся тем же объектом предметной области после смены имени или статуса. Две entity с одинаковыми текущими полями не становятся одной. ORM Identity Map поддерживает один in-memory экземпляр на identity в пределах Unit of Work, но бизнес-равенство всё равно должно быть выражено явно.

Value Object определяется структурным значением всех полей, не имеет самостоятельной identity и обычно immutable: два одинаковых DateRange эквивалентны независимо от экземпляра. Email, DateRange и CustomerId убирают primitive obsession, несут инварианты и не позволяют перепутать одинаковые scalar-типы.

Операция над Value Object возвращает новое значение, а Entity меняет состояние через методы, сохраняя identity и инварианты. Не следует превращать каждую строку в класс: тип оправдан при инварианте, поведении, единицах измерения или риске смешения. ORM mapping и сериализация обязаны сохранять эту семантику.

PHP

```

1 final readonly class CustomerId
2 {
3     public function __construct(public string $value) {}
4
5     public function equals(self $other): bool
6     {
7         return $this->value === $other->value;
8     }
9 }
10
11 final class Customer
12 {
13     public function __construct(
14         private CustomerId $id,
15         private string $name,
16     ) {}
17
18     public function sameIdentityAs(self $other): bool
19     {
20         return $this->id->equals($other->id);
21     }
22 }

```

Слабый сигнал: Сравнение Entity только по набору текущих mutable-полей; Value Object с сеттерами и временно невалидным состоянием; Использование object identity === вместо доменного equals(); Один и тот же тип идентификатора для разных агрегатов.

44. Где размещать Repository, Application Service и Domain Service?

Ответ: Application Service координирует use case: авторизацию, транзакционную границу, загрузку агрегатов, вызов доменного поведения и публикацию результата. Он не должен содержать сложное бизнес-решение, но именно здесь допустимы порты к хранилищу, clock, gateway и unit of work.

Repository — коллекцеподобный порт для агрегатов, обычно определенный рядом с domain/application, а реализованный инфраструктурой. Он оперирует доменными объектами и бизнес-идентификаторами; методы вроде save(Order) или get(OrderId) лучше, чем универсальный findBy(array), протекающий деталями ORM.

Domain Service нужен для правила, которое естественно не принадлежит одной entity/value object и остается чистым либо зависит от узкого доменного порта. Его не следует использовать как корзину процедурного кода. Запросные модели и отчеты могут обходить repositories агрегатов и читать DTO напрямую.

```
1  final readonly class ConfirmOrderHandler
2  {
3      public function __construct(
4          private OrderRepository $orders,
5          private TransactionManager $tx,
6      ) {}
7
8      public function __invoke(ConfirmOrder $command): void
9      {
10         $this->tx->run(function () use ($command): void {
11             $order = $this->orders->get($command->orderId);
12             $order->confirm();
13             $this->orders->save($order);
14         });
15     }
16 }
```

PHP

Слабый сигнал: Generic repository, раскрывающий QueryBuilder всему приложению; Application Service с правилами, дублирующимися между use cases; Domain Service как статический набор несвязанных helpers.

45. Чем доменные события отличаются от интеграционных и как публиковать их надежно?

Ответ: Доменное событие описывает значимый факт внутри bounded context и может быть богато доменными типами. Интеграционное событие — внешний стабильный контракт: оно версионизируется, не раскрывает внутреннюю модель и содержит достаточно данных либо ссылку на доступный публичный ресурс.

Публикация в брокер после COMMIT имеет окно потери, а до COMMIT — риск события о несуществующем изменении. Transactional outbox записывает бизнес-изменение и сообщение одной транзакцией. Relay короткой транзакцией атомарно арендует порцию записей, после COMMIT публикует их, затем идемпотентно отмечает published_at; повтор после падения неизбежен, поэтому consumer обязан быть идемпотентным.

Преобразование domain event в integration event обычно выполняется на границе модуля. Нужны event id, occurred_at, тип, версия схемы, correlation/causation id и наблюдаемая задержка outbox. Удалять записи следует после гарантированного retention, а не сразу после сетевого ACK без аудита.

SQL

```

1 BEGIN;
2 WITH changed AS (
3     UPDATE orders SET status = 'confirmed'
4     WHERE id = :order_id AND status = 'draft'
5     RETURNING id
6 )
7 INSERT INTO outbox(event_id, topic, event_type, payload, occurred_at)
8 SELECT :event_id, 'orders', 'OrderConfirmed.v1', :json, now()
9 FROM changed;
10 COMMIT;
11
12 -- Отдельная короткая транзакция relay:
13 BEGIN;
14 WITH picked AS (
15     SELECT id FROM outbox
16     WHERE published_at IS NULL
17     AND (claimed_until IS NULL OR claimed_until < now())
18     ORDER BY id FOR UPDATE SKIP LOCKED LIMIT 100
19 )
20 UPDATE outbox o
21 SET claimed_by = :worker,
22     claimed_until = now() + interval '30 seconds'
23 FROM picked WHERE o.id = picked.id
24 RETURNING o.*;
25 COMMIT;
26 -- Публикация после COMMIT; published_at обновляется
27 -- идемпотентно с проверкой claimed_by.

```

Слабый сигнал: Синхронная публикация в брокер как часть DB-транзакции без общей транзакции; Интеграционное событие, сериализующее внутреннюю ORM entity; Отсутствие event id и дедупликации у consumer.

46. Как устроить модульный монолит, чтобы он не превратился в распределенный комок внутри одного процесса?

Ответ: Модульный монолит разделяет код по бизнес-возможностям, а не по техническим слоям всего приложения. Каждый модуль владеет моделью, таблицами и публичным API; доступ к его внутренним классам и таблицам из соседнего модуля запрещен. Один deploy и одна база упрощают операции, но логические границы остаются явными.

Синхронное взаимодействие идет через узкие application-порты, асинхронное — через события. Даже при общей физической БД внешние ключи и JOIN между приватными схемами создают сильную связанность; допустимые исключения фиксируют как архитектурный долг с владельцем и сроком.

Границы обеспечиваются автоматически: namespace rules, Deptrac/PHPStan, отдельные Composer-пакеты или тесты импортов. Модуль извлекают в сервис только при независимой нагрузке, cadence команды, надежности или владении данными; сетевой hop не исправляет плохую границу.

PHP

```

1 // Публичный контракт модуля Billing
2 interface BillingApi
3 {
4     public function reserve(ReserveFunds $command): ReservationId;
5 }
6
7 // Shipping зависит только от контракта, а не от Billing\Infrastructure
8 final readonly class StartShipmentHandler
9 {
10     public function __construct(private BillingApi $billing) {}
11 }

```

Слабый сигнал: Общий каталог Entities, используемый всеми модулями; Прямые запросы к приватным таблицам соседнего модуля; Выделение микросервиса только ради модного стека.

47. Когда CQRS оправдан и какую консистентность он добавляет?

Ответ: CQRS разделяет модель команд, защищающую инварианты, и модели чтения, оптимизированные под конкретные представления. Разделение может быть только на уровне классов и SQL в одной базе; отдельные хранилища и брокер — необязательная, более дорогая стадия.

Подход полезен, когда форма чтения резко отличается от модели записи, запросов намного больше, нужны несколько денормализованных проекций или сложная доменная модель. Для обычного CRUD он часто добавляет дублирование DTO, handlers и инфраструктуры без измеримой пользы.

Асинхронные проекции создают eventual consistency: после команды read model некоторое время отстает. UI должен показывать подтвержденный результат команды, pending state или читать authoritative model; projection handler должен быть идемпотентным, хранить checkpoint и уметь перестраиваться. Lag становится частью SLO.

Слабый сигнал: Отдельная база и брокер как обязательное определение CQRS; Отсутствие UX-сценария для отстающей проекции; Read model, от которой зависит инвариант команды.

48. Какие trade-offs у Event Sourcing и как эволюционировать события?

Ответ: В Event Sourcing источником истины является последовательность неизменяемых событий, а текущее состояние агрегата получается replay. Это дает аудит, временные срезы и новые проекции, но усложняет модель, эксплуатацию, отладку, требования к порядку и восстановлению.

Optimistic concurrency проверяет expected stream version; конфликт означает, что команда принимала решение по устаревшему состоянию и должна быть пересчитана, а не слепо сохранена. Длинные потоки ускоряют snapshots, но snapshot — производный кеш: его версия и возможность полного replay обязательны.

Опубликованное событие не переписывают. Используют schema version, tolerant readers, upcaster при чтении или новое компенсирующее событие. Перед выбором Event Sourcing проверяют, действительно ли бизнесу нужен полный журнал решений: обычная state model плюс audit/outbox часто дешевле и надежнее.

Слабый сигнал: Редактирование исторических событий in place; Blind append без expected stream version; Использование Event Sourcing только ради аудита CRUD.

49. Как обрабатывать ambiguous outcome, webhooks и reconciliation внешнего провайдера?

Ответ: Timeout после отправки запроса означает unknown outcome: провайдер мог применить операцию, а ответ потерялся. Каждому действию дают стабильный operation/idempotency key, сохраняют состояние pending_unknown и сначала запрашивают статус у провайдера либо ждут webhook; слепой повтор списания без idempotency contract недопустим.

Webhook проверяют по raw body до JSON parsing: timestamp/nonce, допустимое окно времени и подпись сравнивают constant-time. Event id фиксируют в inbox с UNIQUE constraint. События могут прийти повторно и не по порядку, поэтому локальная state machine применяет только допустимые монотонные переходы, а raw reference сохраняют для аудита без секретов.

Периодический reconciliation сверяет зависшие и расхожие операции с authoritative status API провайдера, имеет checkpoint, rate limit и очередь manual review. Метрики разделяют transport failure, business rejection, unknown outcome, invalid signature и age pending-операций. Так webhook ускоряет сходимость, но не остаётся единственным источником истины.

PHP

```

1  $raw = $request->getContent();
2  $timestamp = $request->headers->get('X-Timestamp', '');
3  $signature = $request->headers->get('X-Signature', '');
4  if (!ctype_digit($timestamp)
5      || abs(time() - (int) $timestamp) > 300) {
6      throw new StaleWebhook();
7  }
8  $signed = $timestamp . '.' . $raw; // provider-defined format
9  $expected = hash_hmac('sha256', $signed, $webhookSecret);
10
11 if (!hash_equals($expected, $signature)) {
12     throw new InvalidWebhookSignature();
13 }
14
15 $event = json_decode($raw, true, flags: JSON_THROW_ON_ERROR);
16 $db->transactional(function () use (
17     $event, $raw, $inbox, $payments
18 ): void {
19     if ($inbox->acceptOnce($event['id'], hash('sha256', $raw))) {
20         $payments->applyAllowedTransition(
21             $event['operation_id'],
22             ProviderStatus::from($event['status'])
23         );
24     }
25 });

```

Слабый сигнал: Повтор неидемпотентного списания сразу после timeout; Проверка подписи по повторно сериализованному JSON вместо raw body; Подпись без timestamp/nonce и ограниченного replay window; Webhook без дедупликации и проверки порядка переходов; Inbox и state transition в разных транзакциях; Отсутствие reconciliation для зависших pending-операций.

50. Как эволюционно разделять legacy-монолит без большого переписывания?

Ответ: Сначала строят карту поведения: критичные use cases, владельцы данных, зависимости, частота изменений и production-метрики. Characterization tests фиксируют наблюдаемое поведение, а не текущую внутреннюю реализацию. Затем выбирают один вертикальный срез с понятной ценностью и относительно чистой границей.

Strangler pattern перехватывает входящий сценарий и направляет его в новый модуль, оставляя остальные пути в legacy. Anti-corruption layer переводит старые типы и ошибки в новый язык; временное dual write требует reconciliation и четкого источника истины, иначе расхождения становятся постоянными.

Каждый шаг должен быть развертываемым, наблюдаемым и обратимым через feature flag или routing switch. После переноса удаляют старый путь и его данные — параллельные реализации без срока только удваивают стоимость. Микросервис выделяют после стабилизации логической границы, а не для ее поиска.

Слабый сигнал: Полная перепись без инкрементальных релизов; Dual write без сверки и владельца источника истины; Сохранение двух реализаций после завершения миграции.

6. Распределённые системы, highload, очереди и caching

51. Что на практике означают at-most-once, at-least-once и exactly-once delivery?

Ответ: At-most-once допускает потерю, но не повторяет доставку; at-least-once повторяет до подтверждения и поэтому допускает дубликаты. Большинство брокеров и HTTP-retry дают именно at-least-once. Exactly-once обычно действует только внутри ограниченной транзакционной области конкретного брокера и не делает внешнюю БД или email-провайдера атомарными с ним.

Надежный consumer выполняет обработку идемпотентно. Event id записывается в inbox с UNIQUE constraint в той же транзакции, что бизнес-изменение; повторная доставка становится по-ор. Для операций

вида «установить статус» идемпотентность проще, чем для «увеличить счетчик», где нужен operation id или журнал проводок.

ACK отправляют только после durable commit. Если процесс упал после commit до ACK, сообщение придет снова и будет отфильтровано inbox. Срок дедупликации должен покрывать максимальное окно redelivery и ручного replay, а payload с тем же id, но другим hash считается нарушением протокола.

```

1  BEGIN;
2  WITH accepted AS (
3      INSERT INTO consumer_inbox(consumer, event_id, payload_hash)
4      VALUES ('billing', :event_id, :payload_hash)
5      ON CONFLICT (consumer, event_id) DO NOTHING
6      RETURNING 1
7  ), changed AS (
8      UPDATE invoices SET paid_at = :paid_at
9      WHERE id = :invoice_id AND paid_at IS NULL
10     AND EXISTS (SELECT 1 FROM accepted)
11     RETURNING 1
12  )
13  SELECT
14     EXISTS (SELECT 1 FROM accepted) AS accepted,
15     EXISTS (SELECT 1 FROM changed) AS changed;
16  -- Если accepted=false, следующим statement в Read Committed:
17  SELECT payload_hash FROM consumer_inbox
18  WHERE consumer = 'billing' AND event_id = :event_id
19  FOR SHARE;
20  -- Строка обязана существовать, а hash — совпасть.
21  -- accepted && !changed: классифицировать до ACK.
22  COMMIT;
23  -- ACK брокеру только после успешной проверки и COMMIT.

```

Слабый сигнал: Обещание end-to-end exactly-once между брокером и любым внешним API; ACK до фиксации бизнес-изменения; Дедупликация только в памяти процесса.

52. Как проектировать retries, backoff и Dead Letter Queue?

Ответ: Повторять можно только transient failures: timeout, временную недоступность, throttling. Ошибка валидации, несовместимая схема или нарушение инварианта не исчезнут от retry. Классификация должна учитывать код ответа и фазу операции: timeout после отправки не сообщает, выполнил ли сервер запрос, поэтому нужен idempotency key.

Используют exponential backoff с full jitter, лимитом попыток и общим time budget. Jitter предотвращает синхронный retry storm, а Retry-After поставщика имеет приоритет. Circuit breaker останавливает бесполезные вызовы к деградировавшей зависимости, но должен иметь bounded half-open probes.

DLQ — карантин, не архив забытых сообщений. Для него нужны alert, владелец, причина, исходный payload, trace context и безопасная процедура replay после исправления. Poison message не должен блокировать всю partition; бесконечный redelivery маскирует инцидент и сжигает ресурсы.

Слабый сигнал: Retry всех исключений без классификации; Одинаковая задержка без jitter для тысяч workers; DLQ без мониторинга и процедуры безопасного replay.

53. Как обеспечить порядок сообщений и параллелизм в очереди?

Ответ: Глобальный порядок резко ограничивает throughput и обычно не нужен. Практическая гарантия — порядок внутри partition, выбранной по business key: события одного order_id идут последовательно, а разные заказы обрабатываются параллельно. Число partitions задает верхнюю границу параллелизма consumer group.

Даже при ordered log повторы, rebalance и несколько producers могут дать неожиданный порядок для бизнес-состояния. События несут aggregate_id и монотонную version; consumer применяет следующую версию, игнорирует уже обработанную и отправляет gap на retry/reconciliation. Timestamp для порядка ненадежен из-за clock skew.

Горячий ключ создает перегруженную partition, поэтому shard key выбирают по распределению нагрузки. Разбить один агрегат на разные partition можно только если его операции коммутативны или существует отдельный механизм сериализации; иначе throughput этого ключа принципиально последовательный.

Слабый сигнал: Требование глобального порядка без бизнес-обоснования; Partition key со значением-константой или hotspot tenant; Использование wall-clock timestamp как единственного номера версии.

54. Какие cache patterns использовать и как бороться со stampede?

Ответ: Cache-aside прост: приложение читает кеш, при miss загружает источник и записывает значение. Он допускает stale data и окно гонки при invalidation; write-through повышает согласованность пути записи, но связывает latency с кешем, а write-behind рискует потерей и переупорядочиванием изменений.

TTL выбирают из допустимой устарелости и стоимости восстановления, добавляя jitter, чтобы ключи не истекали одновременно. От stampede помогают single-flight/lock с коротким lease, stale-while-revalidate и probabilistic early refresh. От penetration несуществующих ключей — negative caching с коротким TTL или Bloom filter.

Инвалидация должна быть привязана к версии данных: ключ вида product:{id}:v{version} избегает удаления чужого нового значения. Кеш — не источник истины; нужны метрики hit ratio, miss latency, eviction и cardinality, а также поведение при полном отказе Redis, чтобы fallback не обрушил базу.

```

1  $cached = $redis->get($key);
2  if ($cached !== false) {
3      return ProductView::fromJson($cached);
4  }
5
6  $lockKey = "lock:$key";
7  if ($redis->set($lockKey, $token, ['NX', 'PX' => 3000])) {
8      try {
9          $value = $repository->getView($id);
10         $redis->setex($key, 300 + random_int(0, 60), $value->toJson());
11         return $value;
12     } finally {
13         // Lua compare-and-delete: удалить lock только если token совпадает
14         $locks->release($lockKey, $token);
15     }
16 }
17 return $staleStore->get($key); // bounded stale fallback

```

PHP

Слабый сигнал: Одинаковый TTL для миллионов одновременно созданных ключей; Удаление lock без проверки token владельца; Неограниченный fallback в БД при падении кеша.

55. Почему распределенной блокировки с TTL недостаточно и что такое fencing token?

Ответ: Lease с TTL может истечь, пока владелец остановлен GC, scheduler pause или сетью. Другой процесс получит lock, а старый после паузы продолжит запись — одновременно действуют два «владельца». Продление lease уменьшает вероятность, но не доказывает взаимное исключение при partitions и долгих паузах.

Fencing token — монотонно растущий номер, выдаваемый при каждом захвате. Защищаемый ресурс принимает операцию только с token больше последнего принятого, поэтому поздняя команда старого владельца отвергается. Проверка должна находиться в самом хранилище или сервисе ресурса, иначе token остается декоративным.

Если операция локальна одной БД, row lock, advisory lock или optimistic version обычно надежнее внешнего Redis-lock. Распределенный lock применяют лишь после формулировки failure model, требований к availability и поведения при потере quorum; для фоновых работ нередко достаточно идемпотентности и уникального claim.

SQL

```

1  -- :token монотонно выдан координатором блокировки
2  UPDATE protected_jobs
3  SET result = :result, last_fencing_token = :token
4  WHERE id = :job_id
5       AND COALESCE(last_fencing_token, 0) < :token
6  RETURNING id, last_fencing_token;
7
8  -- Нет возвращённой строки: id отсутствует либо token устарел;
9  -- эти исходы различают отдельной проверкой/контрактом.

```

Слабый сигнал: Уверенность, что TTL сам по себе гарантирует mutual exclusion; Unlock по имени ключа без уникального token; Fencing token, который защищаемое хранилище не проверяет.

56. Как выбрать алгоритм rate limiting для распределенного API?

Ответ: Fixed window прост, но допускает двойной burst на границе окна. Sliding log точен, но хранит каждое событие; sliding window counter дает приближение дешевле. Token bucket допускает контролируемый burst при заданной средней скорости, а leaky bucket сглаживает выходной поток.

Ключ лимита выбирают по угрозе и бизнесу: user, API key, tenant, IP или endpoint, часто в иерархии. В распределенной системе check-and-increment должен быть атомарным, например Lua в Redis; локальные лимитеры быстрее, но дают общий лимит лишь приблизительно и требуют деления quota между instances.

Нужно определить fail-open или fail-closed при отказе limiter, возвращать 429 и Retry-After, защищать cardinality ключей и доверенные proxy headers. Rate limit ограничивает частоту, но не заменяет concurrency limit и backpressure: десять медленных запросов могут исчерпать pool даже при низком RPS.

LUA

```

1  -- Атомарный fixed window; для burst-sensitive API лучше token bucket
2  local current = redis.call('INCR', KEYS[1])
3  if current == 1 then
4      redis.call('PEXPIRE', KEYS[1], ARGV[1])
5  end
6  if current > tonumber(ARGV[2]) then
7      return {0, redis.call('PTTL', KEYS[1])}
8  end
9  return {1, tonumber(ARGV[2]) - current}

```

Слабый сигнал: Неатомарные GET и INCR в разных командах; Доверие X-Forwarded-For от любого клиента; Один глобальный ключ, создающий hotspot и шумных соседей.

57. Как строить backpressure и graceful degradation в highload-системе?

Ответ: Когда входной поток выше пропускной способности, неограниченная очередь только переносит отказ во времени и увеличивает latency. Система должна ограничивать concurrency у каждого дефицитного ресурса: DB pool, downstream API, CPU worker. Закон Литтла связывает concurrency, throughput и latency, поэтому рост очереди — ранний сигнал насыщения.

Запрос получает общий deadline, а downstream-вызов — меньший timeout из оставшегося budget. Bounded queue, bulkhead, circuit breaker и load shedding защищают полезную работу; retry учитывается в том же budget, иначе он усиливает перегрузку. Важные и фоновые операции разделяют по pool и приоритетам.

Graceful degradation проектируют как продуктовый сценарий: stale read, упрощенный ответ, отключение рекомендаций или 503 с Retry-After. Нельзя молча возвращать ложный успех. Capacity-тесты должны моделировать медленную зависимость, retry storm и восстановление, а метрики — saturation, queue age и rejected work.

Слабый сигнал: Неограниченная in-memory очередь; Timeout каждого downstream больше deadline входящего запроса; Retry storm при уже насыщенном dependency.

58. Как применять CAP и quorum-модель без упрощения «выбрать любые два»?

Ответ: CAP рассматривает поведение во время network partition: система либо сохраняет линейризуемую согласованность, отклоняя часть операций, либо остается доступной и допускает расхождение. В нормальной сети одновременно обсуждают latency, consistency model, durability и failure detection; формула «любые два из трех» скрывает эти детали.

В реплицированном хранилище N , W и R описывают число копий для записи и чтения. Условие $W + R > N$ дает пересечение quorum, но не гарантирует линейризуемость без корректного разрешения версий, координации и обработки sloppy quorum. Last-write-wins по физическим часам теряет данные при clock skew.

Модель выбирают на уровне операции. Остаток средств и уникальное имя требуют сильной координации; счетчик просмотров или presence могут сойтись асинхронно через CRDT/merge. Клиенту также могут быть нужны session guarantees: read-your-writes и monotonic reads, даже если глобальная линейризуемость слишком дорога.

Слабый сигнал: Объяснение CAP как постоянного выбора любых двух свойств; Утверждение, что $W + R > N$ автоматически дает linearizability; Last-write-wins без учета clock skew и потерянных обновлений.

59. Как передавать trace context и причинность через HTTP, очереди и batch-сообщения?

Ответ: Через HTTP распространяют стандартный W3C traceparent/tracestate, а библиотека создаёт child span только после валидации remote context. В message envelope отдельно хранят trace context, correlation_id всего бизнес-процесса и causation_id непосредственного события; эти идентификаторы не подменяют уникальный message_id и idempotency key.

Producer inject-ит context в headers, consumer extract-ит его и создаёт span обработки. При долгой задержке очереди producer span уже завершён — это нормально; queue time фиксируют отдельным атрибутом. Для batch, fan-in и retry причинность не всегда дерево: span links точнее ложного parent-child, а новый retry span связывают с исходной попыткой.

Недоверенный внешний trace_id не используют для авторизации и не дают ему бесконтрольно влиять на sampling. Payload и секреты в spans не пишут; бизнес-идентификаторы нормализуют и ограничивают cardinality. Tail sampling видит только spans, доставленные collector: если он должен выбирать ошибки и медленные цепочки, SDK на этом пути настраивают на AlwaysOn или эквивалентную доставку, а collector — на достаточный буфер и ресурсы.

```
1  {
2    "message_id": "01J...",
3    "correlation_id": "checkout-8421",
4    "causation_id": "payment-authorized-17",
5    "traceparent": "00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01",
6    "occurred_at": "2026-07-25T02:15:00Z",
7    "payload": {"order_id": 8421}
8  }
```

JSON

Слабый сигнал: Один request_id вместо message_id, correlation_id и causation_id; Продолжение недоверенного trace context без валидации; Parent-child для любого fan-in или retry независимо от причинности; Полный payload и токены в span attributes.

60. Как реализовать Saga и выбрать между orchestration и choreography?

Ответ: Saga разбивает бизнес-процесс на локальные транзакции с компенсирующими действиями. Компенсация не является rollback: отправленное письмо не «отправить назад», возврат платежа может завершиться позже, а отмена брони обязана быть идемпотентной. Состояния pending, compensating и manual_review становятся частью доменной модели.

В orchestration координатор явно хранит состояние, таймеры и команды, поэтому поток проще наблюдать и менять, но orchestrator может стать сильно связанным центром. Choreography связывает сервисы событиями и удобна для короткой реакции, однако длинная цепочка быстро становится неявной, циклической и трудной для расследования.

Каждый шаг имеет operation id, inbox/outbox, timeout и политику retry; состояние saga меняется с optimistic version. Нужны audit trail, DLQ/manual repair и тесты падения после каждого commit. Двухфазный commit редко приемлем между автономными сервисами, но Saga выбирают не автоматически: иногда правильнее изменить границу данных и сделать операцию локально атомарной.

SQL

```

1 BEGIN;
2 WITH advanced AS (
3     UPDATE sagas
4     SET state = 'payment_reserved', version = version + 1
5     WHERE saga_id = :saga_id
6     AND state = 'started'
7     AND version = :expected_version
8     RETURNING saga_id
9 )
10 INSERT INTO outbox(event_id, topic, event_type, payload)
11 SELECT :event_id, 'inventory.commands',
12        'ReserveInventory.v1', :payload
13 FROM advanced;
14 COMMIT;
15
16 -- UNIQUE(event_id); пустой advanced не создаёт команду.

```

Слабый сигнал: Компенсация, считающаяся мгновенным и безошибочным rollback; Saga без persisted state, idempotency и manual repair; Длинная choreography-цепочка без единого наблюдаемого процесса.

7. Безопасность

61. Как вы проводите threat modeling для PHP-сервиса перед релизом?

Ответ: Я начинаю не со списка уязвимостей, а с активов и границ доверия: персональные данные, деньги, административные операции, внешние провайдеры, очередь, БД и браузер. На диаграмме потоков отмечаю точки входа, идентичность вызывающего, смену уровня доверия и места хранения, после чего прохожу STRIDE и abuse cases: подмена webhook, повтор платежа, массовый перебор идентификаторов, утечка через экспорт.

Риски ранжирую по вероятности и ущербу, назначаю владельца и проверяемую меру: авторизация на объект, подпись и replay-защита webhook, лимиты, журнал аудита, шифрование, алерт. Модель должна завершаться тестами и критериями релиза; документ, который не влияет на backlog, архитектуру и мониторинг, пользы не приносит.

YAML

```

1 threat: forged_payment_webhook
2 asset: order_state
3 controls:
4   - verify_hmac_before_json_parse
5   - reject_timestamp_older_than_300s
6   - unique_provider_event_id
7 detection: alert_on_signature_failures_rate

```

Слабый сигнал: Ограничиваться сканером OWASP без анализа бизнес-сценариев; Не назначать владельцев и способ проверки мер; Считать внутреннюю сеть доверенной по умолчанию.

62. Почему prepared statements не решают SQL injection полностью и как безопасно строить динамические запросы?

Ответ: Параметры защищают только значения. Имена колонок, направление сортировки, оператор, имя таблицы и фрагменты SQL нельзя передать bind-параметром, поэтому их выбирают из жёсткого allowlist. Для списков IN создают отдельный placeholder на каждый элемент; эмуляцию prepared statements в PDO обычно отключают, чтобы поведение определял драйвер.

Я также проверяю второй порядок инъекции: опасная строка могла ранее попасть в БД и позже быть склеена в административный запрос. Репозитории возвращают типизированные данные, доступ к БД имеет минимальные привилегии, а негативные тесты включают сортировку, фильтры, JSON-path и полнотекстовый поиск — места, где разработчики чаще переходят к конкатенации.

PHP

```

1  $allowedSort = ['created_at', 'total', 'status'];
2  $sort = in_array($input->sort, $allowedSort, true)
3      ? $input->sort : 'created_at';
4  $direction = $input->desc ? 'DESC' : 'ASC';
5  $sql = "SELECT id, total FROM orders"
6      . " WHERE tenant_id = :tenant ORDER BY $sort $direction LIMIT :limit";
7  $pdo->setAttribute(PDO::ATTR_EMULATE_PREPARES, false);
8  $stmt = $pdo->prepare($sql);
9  $stmt->bindValue(':tenant', $tenantId, PDO::PARAM_INT);
10 $stmt->bindValue(':limit', min($input->limit, 100), PDO::PARAM_INT);

```

Слабый сигнал: Подставлять ORDER BY или имя таблицы из запроса; Экранировать SQL вручную вместо параметров; Подключать приложение к БД под владельцем схемы.

63. Как системно защищать серверный PHP-интерфейс от XSS?

Ответ: Основная мера — контекстное кодирование в момент вывода, а не при сохранении: HTML-текст, атрибут, URL, JavaScript и CSS требуют разных правил. Twig autoescape закрывает типичный HTML-контекст, но raw, inline-script, HTML в переводах и пользовательские href требуют отдельного ревью. Если продукт разрешает форматированный HTML, его очищают allowlist-санитайзером, а не регулярным выражением.

CSP использую как второй рубеж: nonce или hash для собственных скриптов, запрет object-src и base-uri, сначала режим Report-Only. Не полагаюсь на CSP при DOM XSS и не разрешаю unsafe-inline ради удобства. Для cookie включаю HttpOnly, но помню, что XSS всё равно может выполнять действия от имени пользователя и читать данные со страницы.

PHP

```

1  $nonce = base64_encode(random_bytes(18));
2  $policy = "default-src 'self'; "
3      . "script-src 'self' 'nonce-$nonce'; "
4      . "object-src 'none'; base-uri 'none'; frame-ancestors 'none'";
5  header('Content-Security-Policy: ' . $policy);
6
7  // В шаблоне: <script nonce="{ { csp_nonce } }" src="/app.js"></script>
8  // Пользовательский текст: { { comment } }; не использовать |raw.

```

Слабый сигнал: Очищать ввод один раз и считать его безопасным во всех контекстах; Использовать strip_tags как HTML-санитайзер; Добавлять unsafe-inline в CSP без плана удаления.

64. Когда нужна CSRF-защита и почему SameSite недостаточно?

Ответ: CSRF релевантен там, где браузер автоматически прикладывает credential: session cookie, Basic Auth или клиентский сертификат. Для изменяющих запросов применяю синхронный токен либо double-submit с криптографической привязкой к сессии, проверяю Origin или Referer как дополнительный сигнал и запрещаю изменение состояния через GET.

SameSite=Lax полезен, но не покрывает все браузеры, поддоменные атаки и сценарии, где same-site не означает same-origin. Для чистого API с Bearer-токеном в Authorization CSRF обычно не является главным риском, пока токен не хранится и не отправляется cookie автоматически; зато возрастают риски XSS и утечки токена.

PHP

```

1  if (!isset($_SESSION['csrf'])) {
2      $_SESSION['csrf'] = bin2hex(random_bytes(32));
3  }
4
5  $sent = (string) ($_POST['_csrf'] ?? '');
6  if (!hash_equals($_SESSION['csrf'], $sent)) {
7      http_response_code(403);
8      throw new RuntimeException('Invalid CSRF token');
9  }

```

Слабый сигнал: Считать POST сам по себе защитой; Принимать state-changing GET; Отключать CSRF для всего API без анализа способа аутентификации.

65. Как спроектировать безопасную cookie-сессию и предотвратить session fixation?

Ответ: Идентификатор сессии должен быть случайным, передаваться только в Secure, HttpOnly cookie с подходящим SameSite и никогда через URL. После логина или повышения привилегий ID ротируют. Старую запись не всегда безопасно удалять немедленно: параллельный запрос может потерять сессию, поэтому её помечают `destroyed_at/new_session_id`, дают короткий grace period и затем удаляют атомарно. На сервере задают абсолютный и idle timeout; выход удаляет серверное состояние, а не только cookie. Привязка к полному IP или User-Agent создаёт ложные разлогины и не является надёжной защитой — признаки лучше использовать для risk scoring. Для распределённого хранилища важны TTL, атомарная ротация и отзыв всех privileged-сессий пользователя.

```
1  session.use_strict_mode = 1
2  session.use_only_cookies = 1
3  session.cookie_secure = 1
4  session.cookie_httponly = 1
5  session.cookie_samesite = Lax
6
7  ; После успешной аутентификации:
8  ; session_regenerate_id(false);
9  ; Старый ID пометить destroyed_at/new_session_id,
10 ; принять только в коротком grace period и затем удалить.
```

INI

Слабый сигнал: Не менять session ID после логина; Немедленно удалять старую сессию без учёта параллельных запросов; Передавать session ID в query string; Считать бесконечный Redis TTL допустимым.

66. Как хранить пароли и строить безопасное восстановление доступа?

Ответ: Пароли хеширую `password_hash` с `PASSWORD_ARGON2ID` и параметрами, измеренными на продуктивном классе железа: проверка должна быть дорогой для атакующего, но выдерживать пиковый логин-трафик. Уникальная соль создаётся библиотекой; отдельный реперг имеет смысл только при хранении в KMS и продуманной ротации. После `password_verify` делаю `password_needs_rehash` и обновляю хеш через `compare-and-swap`, чтобы не затереть параллельную смену пароля.

Токен восстановления — случайный одноразовый секрет с коротким TTL; в БД хранится его хеш, ссылка не раскрывает существование аккаунта. После смены пароля инвалидирую активные сессии и ресовегу-токены, отправляю уведомление и журналирую событие без секрета. Rate limit строю по комбинации аккаунта, IP и устройства, чтобы не превратить защиту в простой DoS на конкретного пользователя.

```
1  $options = [
2      'memory_cost' => 65536,
3      'time_cost' => 3,
4      'threads' => 1,
5  ];
6  $hash = $user->passwordHash();
7
8  if (password_verify($candidate, $hash)
9      && password_needs_rehash($hash, PASSWORD_ARGON2ID, $options)) {
10     $newHash = password_hash($candidate, PASSWORD_ARGON2ID, $options);
11     $users->replacePasswordHashIfCurrent($userId, $hash, $newHash);
12     // 0 обновлённых строк — нормальная конкурентная смена пароля.
13 }
```

PHP

Слабый сигнал: Использовать SHA-256 или общий salt для паролей; Хранить ресовегу-токен в открытом виде; Сообщать, существует ли email, через разные ответы или тайминги.

67. Как предотвращать IDOR и ошибки авторизации в multi-tenant приложении?

Ответ: Аутентификация отвечает, кто вызвал сервис, а авторизация — может ли он выполнить конкретное действие над конкретным объектом. Проверку размещаю в application/domain boundary и передаю в репозиторий tenant_id из доверенного контекста, а не из тела запроса. Загрузка order по одному публичному ID с последующей проверкой слишком легко забывается; безопаснее запрос, который сразу ограничен tenant и доступной областью.

Политики должны учитывать роль, владение, состояние объекта и purpose, а административный bypass быть явным и аудируемым. UUID снижает угадываемость, но не заменяет authorization. Тестовая матрица обязана включать horizontal escalation между пользователями и tenant, vertical escalation до администратора и batch/export endpoints.

```

1  SELECT id, status, total
2  FROM orders
3  WHERE public_id = :order_id
4         AND tenant_id = :authenticated_tenant
5         AND (owner_id = :actor_id OR :can_read_all = TRUE)
6  LIMIT 1;
```

SQL

Слабый сигнал: Считать UUID механизмом авторизации; Брать tenant_id из JSON клиента; Проверять роль только на уровне маршрута без проверки объекта.

68. Как безопасно обрабатывать загрузки файлов и пользовательские архивы?

Ответ: Проверяю размер до и во время чтения, определяю тип по содержимому через finfo, разрешаю только необходимые форматы и генерирую серверное имя. Файл сначала попадает в карантин вне web-root, затем проходит антивирус и, для изображений, декодирование с повторным кодированием. Отдача идёт с безопасным Content-Type, Content-Disposition и X-Content-Type-Options: nosniff.

Для ZIP ограничиваю число файлов, суммарный распакованный размер, степень сжатия и глубину; нормализую каждый путь и отвергаю абсолютные пути, .. и symlink. Одного ограничения расширения недостаточно: polyglot-файл, SVG со скриптом и ZIP bomb обходят наивные проверки. Обработку выполняю под отдельным непривилегированным пользователем с лимитами CPU, памяти и времени.

```

1  $mime = (new finfo(FILEINFO_MIME_TYPE))->file($tmpPath);
2  $allowed = ['image/jpeg' => 'jpg', 'image/png' => 'png'];
3  if (!isset($allowed[$mime]) || filesize($tmpPath) > 5_000_000) {
4      throw new DomainException('Unsupported upload');
5  }
6  $name = bin2hex(random_bytes(24)) . '.' . $allowed[$mime];
7  $quarantine = $quarantineDir . DIRECTORY_SEPARATOR . $name;
8  if (!move_uploaded_file($tmpPath, $quarantine)) {
9      throw new RuntimeException('Upload move failed');
10 }
```

PHP

Слабый сигнал: Доверять Content-Type и исходному имени клиента; Хранить загрузки в исполняемом web-root; Распаковывать ZIP без лимитов и проверки путей.

69. Как защищаться от SSRF в интеграциях, preview URL и webhook-конструкторах?

Ответ: Лучший контроль — allowlist известных схем, доменов и портов, а не blacklist строк. После каждого DNS-разрешения проверяю все адреса и разрешаю только public-global; vetted classifier должен учитывать IPv4, IPv6 и IPv4-mapped IPv6. Соединяюсь именно с проверенным адресом, фиксируя корректный Host/SNI, а каждый redirect разрешаю только после полного повторения проверки. Это закрывает DNS rebinding и переход во внутреннюю сеть.

HTTP-клиент получает короткие connect/read timeout, лимит тела, запрет нестандартных протоколов и минимальный egress через проху или firewall. Ответ не должен автоматически интерпретироваться как HTML или XML. Логирую целевой host, итоговый IP, latency и причину отказа, но не URL с credential; всплеск блокировок — полезный сигнал атаки.

PHP

```

1  $target = $urlPolicy->requireHttpsAllowedHost($url);
2  $addresses = $resolver->resolveAll($target->host());
3  if ($addresses === []) {
4      throw new DnsResolutionFailed();
5  }
6
7  foreach ($addresses as $address) {
8      if (!$ipClassifier->isPublicGlobal($address)) {
9          throw new BlockedOutboundAddress($address);
10     }
11 }
12 $ip = $addresses[0];
13 $port = $target->port() ?? 443;
14 $curlIp = str_contains($ip, ':') ? "[$ip]" : $ip;
15
16 $client->request('GET', $target->uri(), [
17     'allow_redirects' => false,
18     'connect_timeout' => 0.5,
19     'timeout' => 3.0,
20     'curl' => [CURLOPT_RESOLVE => [sprintf(
21         '%s:%d:%s', $target->host(), $port, $curlIp
22     )]], // pin IP; исходный host сохраняет Host и TLS SNI
23 ]); // Redirect проходит resolution, проверку и pin заново.

```

Слабый сигнал: Проверять только строку URL до DNS resolution; Проверить IP, но позволить HTTP-клиенту повторно resolve hostname; Разрешать redirects без повторной валидации; Давать приложению неограниченный сетевой egress.

70. Как управлять секретами, шифрованием данных и ротацией ключей?

Ответ: Секреты не хранятся в Git, образе, exception message или общей переменной CI. Приложение получает их на runtime из secret manager через workload identity; доступ разделён по сервисам и окружениям, выдача аудируется. Ротация должна поддерживать период перекрытия: чтение старым и новым ключом, запись только новым, затем фоновое перешифрование.

Для данных использую стандартное authenticated encryption, например libsodium XChaCha20-Poly1305, с уникальным nonce и контекстом в associated data. В записи храню версию ключа, nonce и ciphertext; data-encryption keys оборачиваются ключом KMS. Сначала определяю угрозу: шифрование столбца не спасает, если скомпрометирован процесс приложения с доступом к KMS.

PHP

```

1  $nonce = random_bytes(SODIUM_CRYPTO_AEAD_XCHACHA20POLY1305_IETF_NPUBBYTES);
2  $aad = 'customer-email:v1:' . $tenantId;
3  $ciphertext = sodium_crypto_aead_xchacha20poly1305_ietf_encrypt(
4      $plaintext, $aad, $nonce, $dataKey
5  );
6  $record = [
7      'key_version' => $keyVersion,
8      'nonce' => base64_encode($nonce),
9      'ciphertext' => base64_encode($ciphertext),
10 ];
11 sodium_memzero($dataKey);

```

Слабый сигнал: Изобретать собственную криптографию или повторно использовать nonce; Хранить ключ рядом с ciphertext в той же БД; Проводить ротацию мгновенным удалением старого ключа.

8. Тестирование, статический анализ и качество

71. Как вы строите тестовую стратегию для крупного PHP-сервиса?

Ответ: Стратегия начинается с рисков и архитектурных границ, а не с процента coverage. Чистую доменную логику покрываю быстрыми unit-тестами, адаптеры БД, брокера и HTTP — integration-тестами на реальных совместимых сервисах, критические пользовательские пути — небольшим числом end-to-end. Контракты между командами проверяю отдельно, чтобы E2E не были единственным сигналом совместимости.

Для каждого слоя определяю скорость, владельца, стабильность данных и диагностичность падения. В pull request запускается быстрый набор, более тяжёлые сценарии — параллельно и перед релизом; production synthetic checks дополняют, но не заменяют тесты. Coverage использую для поиска слепых зон и diff coverage, а не как цель, провоцирующую бессмысленные assertions.

Слабый сигнал: Требовать 100% coverage без анализа рисков; Проверять всю систему только E2E-тестами; Мокать БД или брокер и называть это integration-тестом.

72. Что делает unit-тест устойчивым, а моки — опасными?

Ответ: Устойчивый тест проверяет наблюдаемое поведение и инвариант через публичный контракт, контролирует часы, случайность и идентификаторы, не зависит от порядка запуска. Я мокирую внешнюю границу с важным interaction contract — например, платёжный порт, — но не внутренние классы реализации. Иначе рефакторинг без изменения поведения ломает десятки тестов.

Для value objects и алгоритмов предпочитаю реальные объекты, для времени — Clock, для UUID — генератор, для сложного входа — test data builder. Проверяю не только happy path, но и отсутствие нежелательного эффекта: при отклонённом платеже заказ не подтверждён и событие не опубликовано. Один тест должен объяснять одну бизнес-причину падения.

```
1 public function testExpiredReservationCannotBeConfirmed(): void
2 {
3     $clock = new FrozenClock(new DateTimeImmutable('2026-07-25T12:00:00Z'));
4     $order = OrderBuilder::reservedUntil('2026-07-25T11:59:59Z')->build();
5
6     try {
7         $order->confirm($clock->now());
8         self::fail('Expected ReservationExpired');
9     } catch (ReservationExpired) {
10         // Ожидаемый доменный отказ.
11     }
12
13     self::assertSame(OrderStatus::Reserved, $order->status());
14 }
```

PHP

Слабый сигнал: Мокать каждый collaborator и проверять последовательность внутренних вызовов; Использовать реальное текущее время или sleep; Давать тесту несколько несвязанных причин падения.

73. Как тестировать репозитории, транзакции и миграции БД?

Ответ: Репозиторий тестирую на той же СУБД и близкой версии, что production: SQLite не воспроизводит уровни изоляции, типы, индексы и SQL PostgreSQL/MySQL. Набор поднимает изолированную схему или контейнер, применяет реальные миграции и проверяет ограничения, блокировки, rollback и конкурентные сценарии. Для гонок нужны два независимых соединения, а не последовательные вызовы в одной транзакции.

Откат транзакции после теста быстр, но скрывает поведение commit hooks и фоновых consumers; такие сценарии используют уникальный tenant и явную очистку. Миграции проверяю на копии статистически похожих данных: время блокировки, план запроса и совместимость старой версии приложения с новой схемой являются частью теста.

SQL

```

1  -- Инвариант защищает и от гонки, которую не поймал PHP-код.
2  CREATE UNIQUE INDEX CONCURRENTLY uq_active_reservation
3  ON reservations (resource_id)
4  WHERE released_at IS NULL;
5
6  ALTER TABLE payments
7      ADD CONSTRAINT chk_payment_amount_positive
8      CHECK (amount_minor > 0) NOT VALID;
9  ALTER TABLE payments VALIDATE CONSTRAINT chk_payment_amount_positive;

```

Слабый сигнал: Подменять PostgreSQL на SQLite в integration-тестах; Не тестировать ограничения и конкурентные записи; Считать успешное применение миграции достаточной проверкой.

74. Как тестировать совместимость HTTP API и событий между сервисами?

Ответ: Контракт фиксирует не только форму happy response, но обязательность полей, типы, ошибки, заголовки, семантику idempotency и правила эволюции. Для HTTP применяю OpenAPI validation и consumer-driven contracts, для событий — versioned schema в registry. Provider проверяет реальные handlers и сериализацию, а не дубликат DTO, написанный специально для теста.

Изменения делю на additive и breaking: новое optional-поле обычно совместимо, переименование или изменение смысла — новая версия. Consumers обязаны игнорировать неизвестные поля, а producer сохраняет старую схему на период миграции. Contract-тесты не заменяют небольшой cross-service E2E: они не обнаружат неверную бизнес-последовательность из нескольких корректных вызовов.

JSON

```

1  {
2      "$id": "order.confirmed.v2",
3      "type": "object",
4      "required": ["event_id", "occurred_at", "order_id", "total_minor", "currency"],
5      "properties": {
6          "event_id": {"type": "string", "format": "uuid"},
7          "occurred_at": {"type": "string", "format": "date-time"},
8          "order_id": {"type": "string"},
9          "total_minor": {"type": "integer", "minimum": 0},
10         "currency": {"type": "string", "pattern": "^[A-Z]{3}$"}
11     },
12     "additionalProperties": true
13 }

```

Слабый сигнал: Генерировать provider stub, который всегда соответствует consumer; Запрещать неизвестные поля без необходимости; Менять смысл существующего поля без новой версии.

75. Когда полезны property-based и mutation testing?

Ответ: Property-based testing полезен там, где множество входов, но мало устойчивых инвариантов: сумма распределённых скидок не превышает total, serialize/deserialize сохраняет значение, сортировка идемпотентна, диапазоны не пересекаются. Генератор исследует границы, Unicode, нули и большие значения, а shrinker сводит падение к минимальному примеру.

Mutation testing меняет условия, арифметику и возвращаемые значения и показывает, замечают ли это тесты. Выживший мутант — сигнал слабого assertion или недостижимого кода, но не автоматический дефект; применять лучше к критичной доменной логике и изменённым файлам, иначе стоимость CI велика. Высокий line coverage при низком mutation score разоблачает тесты, которые лишь исполняют код.

Слабый сигнал: Генерировать случайные данные без зафиксированного seed; Требовать одинаковый mutation score для DTO и расчёта денег; Считать каждого выжившего мутанта обязательной доработкой.

76. Как внедрить PHPStan или Psalm в большой legacy-проект без вечного baseline?

Ответ: Сначала фиксирую воспроизводимую версию инструмента, расширения фреймворка и текущий baseline, затем запрещаю новые ошибки на изменённом коде. Уровень повышаю по модулям, начиная с домена и общих библиотек; baseline уменьшается по бюджету и имеет владельца. Игнорирование по шаблону допустимо только с узкой областью и комментарием причины.

Максимальную пользу дают точные сигнатуры на границах: generics для коллекций и репозиторий, array shapes для внешних payload, non-empty-string, positive-int, assert-аннотации и stub-файлы для нетипизированных библиотек. Статический анализ не доказывает бизнес-корректность, но дешево удаляет целый класс null, dead branch и неверных generic-типов до runtime.

```

1  includes:
2    - phpstan-baseline.neon
3
4  parameters:
5    level: 8
6    paths:
7      - src
8    tmpDir: var/phpstan
9    reportUnmatchedIgnoredErrors: true
10   ignoreErrors:
11     - identifier: missingType.iterableValue
12       path: src/Legacy/ImportAdapter.php
13
14   # CI отдельно проверяет, что размер baseline не вырос.
```

NEON

Слабый сигнал: Генерировать новый baseline при каждом падении CI; Игнорировать ошибки широким regex по всему src; Повышать уровень сразу для всего legacy-кода и блокировать команду.

77. Как типы, value objects и DTO улучшают качество PHP-кода?

Ответ: Тип должен сделать некорректное состояние трудно представимым. Money хранит minor units и currency, Email валидирует и нормализует один раз, OrderId не смешивается с UserId, enum ограничивает статус. Это лучше, чем массивы и строки, чьи неявные соглашения приходится помнить каждому caller.

На внешней границе сырой JSON сначала валидируется и преобразуется в входной DTO; домен не должен знать о nullable-полях транспортного формата. При этом не создаю класс на каждую строку без инварианта: типизация имеет стоимость сериализации, маппинга и обучения. Value object оправдан, когда концентрирует правило, единицу измерения или предотвращает смешение.

```

1  final readonly class Money
2  {
3      public function __construct(
4          public int $minor,
5          public Currency $currency,
6      ) {
7          if ($minor < 0) {
8              throw new InvalidArgumentException('Money cannot be negative');
9          }
10     }
11
12     public function add(self $other): self
13     {
14         if ($this->currency !== $other->currency) {
15             throw new CurrencyMismatch();
16         }
17         return new self($this->minor + $other->minor, $this->currency);
18     }
19 }
```

PHP

Слабый сигнал: Передавать деньги как float; Использовать один string для идентификаторов разных сущностей; Создавать value object без инварианта или поведения.

78. Как диагностировать flaky tests и безопасно распараллелить PHP-тесты?

Ответ: Flaky-тест сначала сохраняю как дефект с seed, окружением и историей, затем многократно запускаю в случайном порядке и отдельно. Типичные причины: общая БД или файловая система, статическое состояние, реальное время, незавершённый async job, зависимость от locale и портов. Retry можно временно использовать для сбора данных, но зелёный результат после retry не должен скрывать нестабильность.

Для параллелизма каждый worker получает отдельную schema/database, namespace Redis, queue prefix и временный каталог; фикстуры используют уникальные ключи. Тесты не должны полагаться на автоинкрементные ID и порядок. Quarantine допустим с владельцем и сроком исправления, иначе нестабильность постепенно уничтожает доверие к CI.

```
1 <phpunit executionOrder="random"
2     resolveDependencies="true"
3     failOnRisky="true"
4     failOnWarning="true">
5
6     <php>
7         <env name="APP_ENV" value="test" force="true"/>
8         <env name="CLOCK_MODE" value="frozen" force="true"/>
9     </php>
10 </phpunit>
11 <!-- Worker token добавляется к DB schema и Redis prefix. -->
```

XML

Слабый сигнал: Лечить нестабильность бесконечным retry; Делить одну schema и Redis namespace между workers; Игнорировать seed и порядок запуска при расследовании.

79. Какие quality gates и практики code review уместны для Senior-команды?

Ответ: Автоматизирую однозначное: форматирование, static analysis, unit/integration tests, dependency audit, secret scan, миграционные проверки и diff coverage. Review оставляю для корректности модели, угроз, обратной совместимости, эксплуатации и понятности. Небольшие PR с описанием решения и плана отката проходят быстрее и дают более качественную обратную связь.

Комментарий review должен различать blocker, suggestion и question и объяснять риск, а не личное предпочтение. Для security-, money- и schema-изменений применяю CODEOWNERS или обязательного эксперта, но не создаю очередь из одного человека. Метрики времени review и reopen rate полезнее числа комментариев; цель — снизить дефекты и распространять знания.

Слабый сигнал: Обсуждать стиль вручную при наличии formatter; Мерить качество reviewer количеством комментариев; Сливать огромные PR без описания риска и rollback.

80. Как безопасно рефакторить legacy PHP-код с низким покрытием?

Ответ: Сначала строю seam вокруг изменяемой области и добавляю characterization tests, фиксирующие текущее наблюдаемое поведение, включая странности, от которых зависят клиенты. Снимаю production traces и примеры payload без персональных данных, сравниваю old/new output через shadow execution или approval tests. Только после этого отделяю ввод-вывод от чистой логики и делаю малые преобразования.

Не пытаюсь одновременно переписать архитектуру и изменить бизнес-правила: эти риски должны быть разнесены. Для сложной замены применяю branch by abstraction, feature flag и поэтапный процент трафика с метриками расхождений. Удаление старого пути, флага и временной телеметрии входит в definition of done.

Слабый сигнал: Переписывать модуль целиком без поведенческой сетки; Считать существующее поведение автоматически правильным; Оставлять feature flag и двойную реализацию навсегда.

9. Производительность, наблюдаемость, DevOps и надёжность

81. Как вы расследуете деградацию производительности PHP-приложения?

Ответ: Начинаю с пользовательского симптома и сравнения с baseline: p50/p95/p99 latency, throughput, error rate и saturation по endpoint, tenant и версии. Затем связываю trace с профилем CPU, SQL spans, внешними вызовами, очередью и PHP-FPM wait time. Без этой последовательности легко оптимизировать заметный, но не доминирующий участок.

Для воспроизводимого запроса снимаю sampling-профиль в production с безопасной частотой или в близком стенде, сравниваю flame graph до и после. Проверяю изменение данных и планов БД, cold cache, DNS/TLS, блокировки и рост payload. Оптимизация принимается только с измеримым бюджетом и регрессионным benchmark; среднее время не должно скрывать хвост.

Слабый сигнал: Оптимизировать по интуиции без профиля и метрик; Смотреть только среднюю latency; Проводить нагрузочный тест на несопоставимых данных.

82. Как диагностировать задержки внешнего HTTP API и настроить timeouts, connection reuse и retries?

Ответ: Разбиваю latency на DNS, connect, TLS, time to first byte, download и ожидание свободного connection slot. Trace без этих фаз показывает только «провайдер медленный» и скрывает исчерпанный local pool, отсутствие keep-alive или DNS-проблему. Смотрю p50/p95/p99, status/error class, reuse rate и concurrency по каждому dependency.

Timeout — это бюджет, а не одно число: connect timeout короткий, total/request timeout укладывается в оставшийся end-to-end deadline. Retry разрешён только для классифицированных transient ошибок и идемпотентных операций либо запросов с provider idempotency key; каждая попытка потребляет общий deadline. Использую exponential backoff с jitter, ограничение concurrency и circuit breaker, чтобы деградация зависимости не заняла все PHP workers.

```
1  $remaining = $deadline->remainingSeconds();
2  if ($remaining <= 0) {
3      throw new DeadlineExceeded();
4  }
5  $timeout = min(1.5, $remaining);
6
7  $response = $client->request('POST', '/charges', [
8      'headers' => ['Idempotency-Key' => $operationId],
9      'connect_timeout' => min(0.25, $timeout),
10     'timeout' => $timeout,
11 ]);
12
13 // Retry только для разрешённой операции и пока остаётся общий deadline.
```

PHP

Слабый сигнал: Один большой timeout без connect/total budget; Retry POST без idempotency key и классификации ошибки; Неограниченный параллелизм к деградирующей зависимости; Метрика только общего времени без DNS/TLS/pool фаз.

83. Как расследовать рост RSS и CPU в долгоживущем PHP worker?

Ответ: Сначала различаю PHP allocated/used memory и RSS процесса: allocator удерживает арены, extensions имеют native allocations, OPcache/shared memory не равны heap запроса. Строю график after-job baseline, memory delta по типу сообщения, gc_status, числу managed ORM entities, открытых ресурсов и размеру статических/global caches; единичный snapshot без тренда мало полезен.

Воспроизвожу на одном worker с репрезентативной последовательностью jobs и профилирую allocation/CPU flame graph. Частые причины — Identity Map без clear, listeners/closures в глобальном registry, неограниченный cache, буферы клиента, циклы объектов и блокирующий IO. Сначала откатывают все уровни незавершённой транзакции; при ошибке соединение закрывают и пересоздают. Затем освобождают ресурсы и сбрасывают контексты; ручной GC включают только по профилированию. max_requests — предохранитель, а не исправление.

PHP

```

1  $before = memory_get_usage(true);
2  try {
3      $handler($message);
4  } finally {
5      while ($connection->isTransactionActive()) {
6          $connection->rollBack();
7      }
8      $entityManager->clear();
9      $resources->closeAll();
10     $logger->reset();
11     $requestContext->reset();
12     $container->reset();
13     $metrics->observe('worker.memory_delta_bytes',
14         memory_get_usage(true) - $before
15     );
16 }

```

Слабый сигнал: Считать `memory_get_usage()` равным RSS процесса; Лечить проблему только снижением `max_requests`; Один profiler snapshot без серии одинаковых jobs; Не сбрасывать ORM, logging context и scoped services между сообщениями.

84. Как capacity-plan-ить очередь и масштабировать consumers, не перегружая downstream?

Ответ: Основные сигналы — arrival rate, service rate на consumer, queue lag, age старейшего сообщения, in-flight, redelivery и доля ошибок. По Little's Law оцениваю требуемую concurrency, но подтверждаю load test: среднее скрывает тяжёлые типы jobs и tail latency. Scaling по длине очереди без age и throughput может дёргаться либо опаздывать.

Максимум consumers ограничен пропускной способностью БД и внешних API. Prefetch задают на pod, а общий бюджет concurrency для всех pod обеспечивают распределённым semaphore/rate limiter; локальный лимит каждого экземпляра не защищает downstream при autoscaling. Autoscaler имеет cooldown и запас на cold start; poison messages быстро уходят в DLQ, иначе retry storm создаёт ложную потребность в масштабировании.

YAML

```

1  autoscaling:
2    metric: queue_oldest_message_age_seconds
3    target: 30
4    min_replicas: 2
5    max_replicas: 40
6    scale_up_cooldown: 15s
7    scale_down_cooldown: 180s
8  downstream:
9    global_concurrency_budget: 96
10   enforcement: distributed_semaphore
11  consumer:
12    prefetch_per_pod: 8

```

Слабый сигнал: Масштабирование только по queue length; max replicas без учёта connection pool и rate limit downstream; Очень большой prefetch ради throughput; Retry storm, интерпретированный как полезная входящая нагрузка.

85. Когда выбирать PHP-FPM, а когда RoadRunner/Swoole/Octane, и какие lifecycle-риски создаёт long-running runtime?

Ответ: FPM даёт сильную request isolation и простую эксплуатационную модель; это хороший default, если bootstrap не доминирует и не нужна event-loop concurrency. Long-running runtime амортизирует boot/container compilation, уменьшает latency и может держать соединения, но меняет семантику процесса: globals, statics, singletons и зарегистрированные listeners переживают запрос.

Переход оправдываю benchmark конкретного workload и operational readiness, а не обещанием «быстрее PHP». Все request/tenant/security contexts имеют scoped lifetime и reset, mutable singleton запрещён, а все уровни leaked transaction откатывают после полной отправки ответа, включая stream; при ошибке rollback

соединение закрывают. Blocking client внутри event loop блокирует всех coroutines; совместимость extensions, signals, memory trend, graceful reload и worker recycling тестируют отдельно.

```

1  try {
2      $requestScope->begin($requestId, $tenantId);
3      $response = $kernel->handle($request);
4      $runtime->emit($response); // ждёт завершения stream
5      return $response;
6  } finally {
7      while ($connection->isTransactionActive()) {
8          $connection->rollBack();
9      }
10     $entityManager->clear();
11     $logger->reset();
12     $requestScope->reset();
13 }

```

PHP

Слабый сигнал: Singleton с текущим tenant или Request; Перенос FPM-кода без теста последовательных запросов в одном worker; Blocking IO внутри event loop без совместимого adapter; Выбор runtime только по synthetic hello-world benchmark.

86. Как построить observability: логи, метрики и distributed tracing?

Ответ: Логи отвечают «что произошло» и содержат структурированные event name, severity, trace_id, request_id, безопасные business identifiers и версию сервиса. Метрики отвечают «насколько часто и насколько плохо»: RED для запросов, USE для ресурсов, бизнес-инварианты для результата. Trace связывает PHP, SQL, брокер и внешние API и показывает критический путь.

Кардинальность контролирую: user_id, URL с ID и exception message не являются labels метрик. Sampling traces оставляет все ошибки и медленные запросы, а нормальные — вероятностно; контекст должен передаваться через HTTP и message headers. Телеметрия проходит редактирование PII и имеет retention, иначе observability становится каналом утечки и неконтролируемой статьёй расходов.

```

1  {
2      "event": "payment_provider_call",
3      "level": "warning",
4      "service": "checkout",
5      "service_version": "2026.07.25-3",
6      "trace_id": "4f7c...",
7      "provider": "acquirer_a",
8      "outcome": "timeout",
9      "duration_ms": 2012,
10     "attempt": 2
11 }

```

JSON

Слабый сигнал: Писать stack trace и payload каждого запроса без редактирования; Добавлять user_id как Prometheus label; Не передавать trace context через очередь.

87. Что такое SLI, SLO и error budget и как на них строить алерты?

Ответ: SLI — измеримая доля хороших событий с точки зрения пользователя, например успешные checkout дешевле 800 мс; SLO — целевая доля за окно, например 99,9% за 28 дней. Error budget — допустимые плохие события. Он превращает спор reliability против скорости в правило: при быстром расходовании бюджета команда приостанавливает рискованные релизы и устраняет источник.

Алерты строю по multi-window multi-burn-rate: быстрый burn ловит серьёзный инцидент, медленный — устойчивую деградацию, оба привязаны к исчерпанию бюджета. CPU сам по себе чаще диагностический сигнал, а не pager. У каждого page есть owner, runbook и ожидаемое действие; neactionable alert удаляется или переводится в dashboard.

PROMQL

```

1  # Доля плохих checkout за 5 минут.
2  sum(rate(http_requests_total{route="checkout",sli="bad"}[5m]))
3  /
4  sum(rate(http_requests_total{route="checkout"}[5m]))
5
6  # Для SLO 99.9% fast burn 14.4x соответствует порогу 0.0144.
7  # Page только если одновременно горят короткое и длинное окна.

```

Слабый сигнал: Называть uptime процесса пользовательским SLI; Пейджить по любому высокому CPU; Создавать alert без runbook и владельца.

88. Как построить release pipeline с canary, feature flags и безопасным rollback?

Ответ: Pipeline один раз собирает immutable artifact, фиксирует lockfiles и SBOM, запускает тесты, static analysis, security scan и smoke-test этого же artifact. Promotion между средами не пересобирает image. Перед rollout проверяются совместимость схемы/событий, миграционный plan, observability и владелец rollback; ручное окно не заменяет эти gates.

Canary получает малую репрезентативную долю traffic или tenant cohort. Решение расширять rollout принимают по заранее выбранным SLI: error rate, latency, бизнес-конверсия и queue lag. Метрики фильтруют по версии и cohort; стадия наблюдения должна полностью покрывать длинное окно. Автоматический rollback требует достаточного числа запросов и одновременного нарушения короткого и длинного burn-rate окна относительно control. Feature flag имеет owner и expiry; откат допустим только при совместимых данных и side effects.

YAML

```

1  canary:
2    traffic_percent: 5
3    observation: 75m
4    metrics_scope:
5      version: canary
6      cohort: canary
7    promote_when:
8      error_rate: "< 0.5%"
9      p95_latency_ms: "< 350"
10     checkout_success_delta: "> -0.3%"
11    rollback_when:
12      min_requests: 1000
13      compare_with: control
14      require_all:
15        burn_rate_5m: "> 14.4"
16        burn_rate_1h: "> 6"

```

Слабый сигнал: Пересборка image для каждой среды; Canary promotion только по CPU и liveness; Feature flag без владельца и даты удаления; Обещание rollback без анализа данных и внешних side effects.

89. Как настраивать контейнерные health probes и resource limits для PHP?

Ответ: Liveness отвечает только на вопрос, завис ли процесс и нужен ли restart; она не должна падать из-за временной недоступности БД, иначе весь кластер войдёт в restart storm. Readiness проверяет способность принимать новый трафик и может учитывать критичную зависимость с коротким timeout. Startup probe даёт прогреться медленному контейнеру, не ослабляя liveness навсегда.

Requests задают планирование, limits защищают соседей, но слишком низкий memory limit создаёт OOM, а жёсткий CPU limit — throttling и хвост latency. Размер pod, число FPM workers, connection pools и HPA должны быть согласованы: масштабирование PHP в десять раз не создаёт десятикратную ёмкость БД. Проверяю graceful termination и PodDisruptionBudget.

YAML

```

1  readinessProbe:
2    httpGet: {path: /health/ready, port: 8080}
3    periodSeconds: 5
4    timeoutSeconds: 1
5  livenessProbe:
6    httpGet: {path: /health/live, port: 8080}
7    periodSeconds: 10
8    failureThreshold: 3
9  resources:
10   requests: {cpu: 500m, memory: 512Mi}
11   limits: {cpu: "2", memory: 768Mi}
12  terminationGracePeriodSeconds: 30

```

Слабый сигнал: Проверять БД в liveness probe; Не согласовывать max_children с memory limit; Масштабировать приложение без лимита соединений к БД.

90. Как проектировать disaster recovery, backup и graceful degradation?

Ответ: Сначала бизнес задаёт RPO — допустимую потерю данных — и RTO — время восстановления; они определяют репликацию, частоту backup и стоимость решения. Backup шифрую, храню независимо от основного аккаунта и защищаю от удаления, но главным тестом является регулярное восстановление в изолированную среду с проверкой целостности и времени. Реплика не заменяет backup: она быстро реплицирует удаление и повреждение.

Для деградации классифицирую зависимости: без каталога можно отдать stale cache, без рекомендаций скрыть блок, без платёжного провайдера принять заказ нельзя. Применяю timeout, bounded retry, circuit breaker, bulkhead и load shedding, сохраняя честный ответ пользователю. Runbook содержит переключение, проверку данных, DNS/credential шаги и ответственного; game day проверяет документ на практике.

Слабый сигнал: Говорить о backup без теста restore; Считать read replica резервной копией; Повторять запросы без лимита и jitter при деградации.

10. Лидерство, инциденты и практический system design

91. Как Senior-разработчик принимает и документирует архитектурные решения?

Ответ: Я формулирую контекст, ограничения и измеримые quality attributes: объём, latency, consistency, RPO/RTO, compliance, навыки команды и срок. Сравниваю минимум два реалистичных варианта, включая «ничего не менять», по стоимости разработки и эксплуатации, failure modes и обратимости. Решение, принятое без данных, помечаю как гипотезу и назначаю spike или метрику.

ADR фиксирует принятое решение, альтернативы, последствия и дату пересмотра, но не превращается в роман. Для дорогого необратимого шага провожу RFC и threat/operability review; для обратимого — делегирую и быстро проверяю в production. Архитектура — не голосование и не власть самого старшего: ответственный принимает решение после сбора возражений и явно объясняет trade-off.

Слабый сигнал: Выбирать технологию по популярности без ограничений задачи; Документировать только плюсы выбранного варианта; Создавать комитет для каждого обратимого решения.

92. Как развивать инженеров через делегирование, mentoring и code review?

Ответ: Делегирую не только реализацию, но контекст, критерии успеха и право принять решение в обозначенных границах. Задача должна быть чуть выше текущего уровня, но иметь безопасные checkpoints; я не забираю её при первой ошибке. После обсуждаем ход мысли и эксплуатационный результат, а не только финальный diff.

Обратная связь конкретна: наблюдение, влияние и следующий шаг; важные темы обсуждаю лично, не прячу их в review. План роста опирается на повторяющиеся примеры и бизнес-вклад, а не на количество строк или громкие проекты. Хороший Senior уменьшает bus factor: документация, парная работа, ротация ownership и возможность команды принимать решения без него — часть результата.

Слабый сигнал: Делегировать ответственность без полномочий и контекста; Исправлять чужую задачу молча вместо обучения; Становиться обязательным reviewer для каждого изменения.

93. Как оценивать крупную техническую инициативу при высокой неопределённости?

Ответ: Разбиваю инициативу на outcomes, вертикальные slices, зависимости и неизвестные. Для исследуемых частей даю диапазон и confidence, а не ложную точность; самые опасные гипотезы проверяю time-boxed spike. Отдельно учитываю migration, observability, security review, rollout, документацию и удаление старого пути — именно их часто забывают в оценке.

План пересматривается после milestones по фактической скорости и найденным рискам. Со стейкхолдерами обсуждаю треугольник score, date и capacity и предлагаю минимальный безопасный вариант, а не скрытый overtime. Прогноз — инструмент решения, не обещание; при изменении данных рано сообщаю новый диапазон, последствия и варианты.

Слабый сигнал: Давать точную дату до исследования неизвестных интеграций; Не включать миграцию и rollout в объём; Компенсировать неопределённость постоянным overtime.

94. Как организовать работу команды во время production-инцидента?

Ответ: Назначаю incident commander, который держит приоритеты, scribe для timeline и технических исполнителей по направлениям. Сначала подтверждаем пользовательский impact и останавливаем ухудшение: rollback, feature disable, traffic shift или load shedding. Глубокий root cause вторичен, пока система не стабилизирована; рискованное изменение проходит явное подтверждение и имеет обратный ход.

Коммуникация идёт по фиксированному cadence: impact, затронутые функции, сделанное, следующий шаг и время следующего обновления. Один канал является источником истины, решения и timestamps записываются. После восстановления проверяем backlog, целостность данных и повторную обработку, лишь затем закрываем инцидент и назначаем post-incident review.

```
1 SEV-1 UPDATE 14:20 UTC
2 Impact: 31% checkout requests fail in EU; started 13:54.
3 Mitigation: new payment route disabled; errors now 4% and falling.
4 Data: no duplicate captures found; reconciliation is running.
5 Next: validate backlog, then restore traffic in 10% steps.
6 Next update: 14:35 UTC.
7 IC: @name Scribe: @name Tech lead: @name
```

TEXT

Слабый сигнал: Одновременно менять production нескольким людям без координации; Искать виновного и root cause до mitigation; Объявлять восстановление до проверки данных и backlog.

95. Как проводить blameless postmortem и выбирать корректирующие действия?

Ответ: Postmortem восстанавливает timeline из логов, deploy events и решений, разделяет trigger, contributing factors и причины длительного обнаружения или восстановления. «Инженер ошибся» ничего не объясняет: нужно понять, почему система позволила одной ошибке пройти review, тесты, rollout и мониторинг. Blameless означает честно исследовать контекст решения, а не избегать ответственности.

Action items конкретны, имеют владельца, срок и проверяемый эффект; приоритет получают меры, сокращающие класс инцидентов: constraint, safe default, canary, автоматический rollback, улучшение detection. Ещё один документ или призыв «быть внимательнее» слабее инженерного барьера. Через срок проверяю выполнение и изменились ли MTTD, MTTR или частота повторения.

Слабый сигнал: Останавливать анализ на человеческой ошибке; Создавать десятки action items без приоритета и владельца; Проводить postmortem без проверки выполнения решений.

96. System design: как спроектировать идиоматичный checkout и обработку платежа?

Ответ: Клиент передаёт idempotency key, связанный с tenant, операцией и хешем запроса. В одной транзакции создаю или читаю запись операции и заказа; повтор с тем же ключом и другим payload отклоняю, с тем же — возвращаю сохранённый результат. Состояние заказа меняется конечным автоматом, денежные суммы хранятся в minor units, а invariant повторно защищён уникальными ограничениями БД.

Вызов провайдера нельзя держать внутри долгой DB-транзакции: команду публикую через transactional outbox, worker вызывает provider с тем же idempotency key и сохраняет результат. Webhook проверяется по подписи, timestamp и provider event_id и может прийти раньше HTTP-ответа. Reconciliation периодически сверяет локальный ledger с провайдером; timeout означает unknown, а не автоматический failure и новый платёж.

```

1  CREATE TABLE idempotency_keys (
2      tenant_id bigint NOT NULL,
3      operation text NOT NULL,
4      key text NOT NULL,
5      request_hash bytea NOT NULL,
6      response_code int NULL,
7      response_body jsonb NULL,
8      expires_at timestampz NOT NULL,
9      PRIMARY KEY (tenant_id, operation, key)
10 );
11
12 CREATE UNIQUE INDEX uq_provider_payment
13 ON payments (provider, provider_payment_id);

```

Слабый сигнал: Повторять платёж после timeout как новую операцию; Хранить idempotency key без хеша исходного запроса; Менять order и публиковать событие двумя независимыми действиями.

97. System design: как построить сервис уведомлений для email, SMS и push?

Ответ: Producer отправляет бизнес-намерение с template_id, locale, recipient reference, variables и deduplication key, но не готовый HTML для каждого канала. Notification service проверяет предпочтения, consent, quiet hours и suppression list, выбирает канал и версию шаблона, затем создаёт отдельные delivery attempts. PII минимизируется, а шаблоны проходят escaping и preview.

Очереди разделяю по каналу и приоритету, provider adapters имеют timeout, retry budget, circuit breaker и rate limit. Permanent bounce, invalid number и unsubscribe не повторяются; transient ошибки уходят в backoff и DLQ. Статус имеет семантику accepted/sent/delivered/failed, webhook провайдера идемпотентен, а метрики показывают delay, delivery rate и suppression — не только число обработанных сообщений.

```

1  {
2      "event_id": "7cebe12d-6e12-4a46-8204-ae7f14161c6",
3      "deduplication_key": "invoice-419-reminder-1",
4      "template_id": "invoice_overdue",
5      "template_version": 3,
6      "recipient_ref": "customer:8421",
7      "locale": "ru-RU",
8      "priority": "normal",
9      "variables": {"invoice_number": "419", "days": 3}
10 }

```

Слабый сигнал: Смешивать транзакционные и маркетинговые сообщения без consent; Повторять permanent bounce; Считать accepted провайдером фактом доставки.

98. System design: как реализовать неизменяемый audit log в multi-tenant системе?

Ответ: Audit event отвечает кто, когда, откуда, над чем и что сделал, содержит outcome, request/trace ID и минимальный diff, но не секреты и лишние персональные данные. Actor включает тип — user, service, support impersonation — и исходную идентичность. Запись критичного события происходит в той же транзакции через outbox или в append-only store; обычный application log не является аудитом.

Доступ отделён от операционной БД, tenant isolation и retention применяются на чтении и хранении. Для каждого tenant append сериализуют по sequence_no; event_hash считают от previous_hash и канонического представления всего события, включая identity, actor, время, действие, outcome и payload.

UNIQUE(tenant_id, sequence_no) и head row предотвращают две конкурирующие вершины цепочки.

Хеш-цепочка только обнаруживает изменение, если доверенный root хранится вне той же БД.

Периодически якорю head или Merkle root в WORM/object-lock хранилище либо внешней подписанной

системе. INSERT-only таблица сама по себе не абсолютная неизменяемость: администратор всё ещё может переписать историю и пересчитать локальные хеши. Экспорт подписывается, а поиск индексируется по tenant, времени, actor и resource.

SQL

```

1 CREATE TABLE audit_events (
2     tenant_id bigint NOT NULL,
3     sequence_no bigint NOT NULL,
4     event_id uuid NOT NULL,
5     occurred_at timestampz NOT NULL,
6     actor_type text NOT NULL,
7     actor_id text NOT NULL,
8     action text NOT NULL,
9     resource_type text NOT NULL,
10    resource_id text NOT NULL,
11    outcome text NOT NULL,
12    trace_id text NOT NULL,
13    payload jsonb NOT NULL,
14    previous_hash bytea NOT NULL,
15    event_hash bytea NOT NULL,
16    PRIMARY KEY (tenant_id, event_id),
17    UNIQUE (tenant_id, sequence_no)
18 );
19
20 -- В транзакции блокируем audit_head tenant-a FOR UPDATE.
21 -- hash = SHA256(previous_hash || canonical_full_event).
22 -- Новый head периодически якорится во внешнем WORM store.
```

Слабый сигнал: Писать токены, пароли или полный request body в аудит; Называть обычный изменяемый application log неизменяемым; Хеш-цепочка без сериализованного sequence и внешнего trusted anchor; Не журналировать impersonation и неудачные privileged-действия.

99. System design: как построить возобновляемый импорт многогигабайтного CSV/XLSX?

Ответ: Upload идёт напрямую в object storage по ограниченному presigned URL; API создаёт Import с tenant, checksum, schema version и idempotency key. Coordinator один раз потоково читает CSV либо SAX-reader для XLSX, нормализует строки и сохраняет детерминированные chunk-артефакты в object storage или staging. Это важно для XLSX: произвольный seek к логической строке ZIP/XML обычно невозможен, а retry не должен перечитывать гигабайты с начала. Queue передаёт только import_id/chunk_id.

Каждый chunk имеет UNIQUE(import_id, chunk_no), lease с worker_token/locked_until/attempt и собственную короткую транзакцию. Просроченный lease можно безопасно забрать повторно; business upserts и перевод chunk в done фиксируют одной транзакцией. Ошибки строк сохраняются отдельно с лимитом примеров, progress считается по подтверждённым chunks.

Нужны quotas, antivirus/content checks, backpressure к БД, pause/cancel, retention исходника и процедура resume. Для XLSX отдельно ограничивают распакованный размер, число entries и compression ratio, иначе небольшой ZIP становится zip bomb. Atomic mode для миллионов строк обычно реализуют через staging table и явный commit/swap после полной проверки.

SQL

```

1  -- Coordinator создал нормализованный immutable artifact:
2  INSERT INTO import_chunk(
3      import_id, chunk_no, object_key, checksum, status
4  ) VALUES (
5      :import_id, :chunk_no, :object_key, :checksum, 'pending'
6  ) ON CONFLICT (import_id, chunk_no) DO NOTHING;
7
8  BEGIN;
9  WITH picked AS (
10     SELECT id FROM import_chunk
11     WHERE status = 'pending'
12         OR (status = 'processing' AND locked_until < now())
13     ORDER BY id FOR UPDATE SKIP LOCKED LIMIT 1
14 )
15 UPDATE import_chunk c
16 SET status = 'processing', worker_token = :token,
17     locked_until = now() + interval '60 seconds',
18     attempt = attempt + 1
19 FROM picked WHERE c.id = picked.id
20 RETURNING c.*;
21
22 -- Здесь же выполняются идемпотентные business upserts,
23 -- затем status='done' с проверкой worker_token.
24 COMMIT;

```

Слабый сигнал: Читать весь XLSX/CSV в память PHP; Передавать файл или тысячи строк внутри queue message; Повторно разбирать весь XLSX с начала для каждого retry chunk; Одна транзакция на весь импорт; Progress по прочитанным, а не подтверждённым строкам; Resume без checksum, lease и idempotency; XLSX без лимита распакованного размера и compression ratio.

100. System design: как эволюционно разделить PHP-монолит без distributed monolith?

Ответ: Начиная с причин: независимое масштабирование, ownership, разные availability или release cadence, а не цель «микросервисы». В монолите выделяю модуль по bounded context, запрещаю прямой доступ к его таблицам через архитектурные тесты и создаю стабильный application API. Strangler переносит один вертикальный поток с observability и сравнением результатов, а не технические слои по очереди.

Новый сервис владеет своими данными; синхронизация идёт через versioned events/outbox, а cross-service transaction заменяется saga и явными compensations. На переходе определяю единственного writer и source of truth. Для обратимого cutover временно поддерживаю reverse sync с checkpoint из нового сервиса в старый; rollback требует остановить новые записи, дождаться отставания старой стороны до нуля и только затем вернуть routing.

Dual write и reverse sync имеют срок удаления, reconciliation и наблюдаемую задержку. События несут origin, монотонную version и event_id; consumer дедуплицирует их, а reverse-sync пишет через путь, который не порождает обратное доменное событие. Иначе old -> new и new -> old образуют feedback loop. Успех измеряю lead time, инцидентами и стоимостью эксплуатации; если граница неустойчива, модульный монолит часто лучше сети.

TEXT

```

1  Phase 1: OrdersModule API + запрет SQL-доступа из других модулей
2  Phase 2: backfill + outbox old -> new; фиксируется checkpoint
3  Phase 3: shadow reads и сравнение результатов
4  Phase 4: один writer; reverse sync с origin/version/event_id
5  Phase 5: routing по tenant cohort + reconciliation
6  Phase 6: после rollback-window удалить reverse sync и старый path
7
8  Rollback: drain writes -> дождаться old checkpoint ->
9  переключить routing -> возобновить writes на старой стороне.

```

Слабый сигнал: Делить монолит по контроллерам, сервисам и репозиториям; Оставлять сервисам общую схему БД; Вводить dual write без source of truth и reconciliation; Reverse sync без origin/dedup, создающий feedback loop; Возвращать routing при rollback до остановки writes и доронки reverse sync.

Основные источники

Технические детали сверены с первичной документацией. Версионный срез: 25 июля 2026 года.

PHP: поддерживаемые версии: <https://www.php.net/supported-versions.php>

PHP Language Reference: <https://www.php.net/manual/en/langref.php>

PHP 8.5: руководство по миграции: <https://www.php.net/manual/en/migration85.php>

Composer: basic usage и composer.lock: <https://getcomposer.org/doc/01-basic-usage.md>

Composer: оптимизация autoloader: <https://getcomposer.org/doc/articles/autoloader-optimization.md>

PHP-FIG: принятые и проектируемые PSR: <https://www.php-fig.org/psr/>

PostgreSQL: Transaction Isolation: <https://www.postgresql.org/docs/current/transaction-iso.html>

PostgreSQL: Using EXPLAIN: <https://www.postgresql.org/docs/current/using-explain.html>

OWASP Cheat Sheet Series: <https://cheatsheetseries.owasp.org/>

Symfony: HttpKernel и request-response lifecycle: https://symfony.com/doc/current/components/http_kernel.html

Laravel: Service Container: <https://laravel.com/docs/container>

OpenTelemetry PHP: <https://opentelemetry.io/docs/languages/php/>